

S.6.19. n sıfırdan büyük bir tamsayı olmak üzere $f(n) = \lceil n / 2 \rceil$ değerini bulan bir Turing makinesi tasarlamamız isteniyor.

$$M_{S.6.19} = \langle Q, \Sigma, \Gamma, \delta, q_0, B, F \rangle$$

$$Q = \{q_0, \dots\}$$

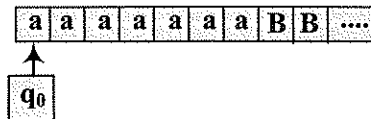
$$\Sigma = \{a\}$$

$$\Gamma = \{a, b, \$, B\}$$

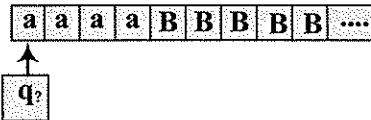
q_0 : Başlangıç durumu.

$$F = \Phi$$

Başlangıç kofigürasyonu ($n = 7$ için)



Bitiş kofigürasyonun ise aşağıdaki gibi olması isteniyor ($n = 7$ için)



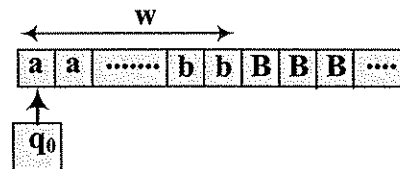
Durumlar kümesini tamamlayıp makinenin hareket çizeneğini anlaşılır biçimde oluşturunuz.

Not: Tasarlayacağınız makinenin yukarıdaki tanımlara (Γ ile başlangıç ve bitiş kofigürasyonlarına) tam uygun olması istenmektedir.

S.6.20. Tanımı aşağıda verilen $L_{S.6.20}$ dilini tanıyan temel model (deterministik) bir Turing makinesi tasarlamamız isteniyor.

$$L_{S.6.20} = \{a^n b^{n+2} \mid n \geq 0\}$$

Başlangıç kofigürasyonu:



$$M_{S.6.20} = \langle Q, \Sigma, \Gamma, \delta, q_0, B, F \rangle$$

$$Q = \{q_0, \dots\}$$

$$\Sigma = \{a, b\}$$

$$\Gamma = \{a, b, X, Y, B\}$$

q_0 : Başlangıç durumu

$$F = \{?\}$$

Makinenin nasıl çalışacağını sözel olarak açıklayınız.

Durumlar kümesini tamamlayıp makinenin hareket çizeneğini anlaşılır biçimde oluşturunuz.

$w_1 = aabbbb$ ve $w_2 = bb$ için tasarladığınız makinenin doğru çalıştığını kontrol ediniz ve bitiş kofigürasyonlarını gösteriniz.

Not: Tasarlayacağınız makinenin yukarıdaki tanımlara (Γ ile başlangıç kofigürasyonuna) tam uygun olması istenmektedir.

S.6.21. Aşağıda tanımı ve geçiş çizeneği verilen Turing makinesi tarafından tanınan $\{a, b, c\}$ alfabesindeki $L_{S.6.21}$ dilinin biçimsel tanımını oluşturunuz.

$L_{S.6.21}$ dilini türeten ve olabildiğince az sözdizim değişkeni bulunan bir dilbilgisi ($G_{S.6.21}$) tanımlayınız.

$$M_{S.6.21} = \langle Q, \Sigma, \Gamma, \delta, q_0, B, F \rangle$$

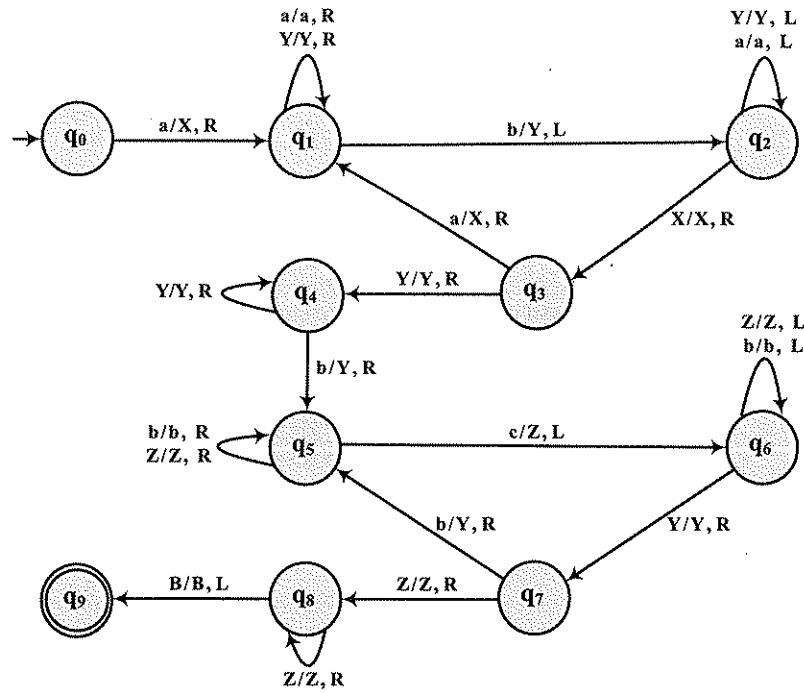
$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9\}$$

$$\Sigma = \{a, b, c\}$$

$$\Gamma = \{a, b, c, X, Y, Z, B\}$$

q_0 : Başlangıç durumu.

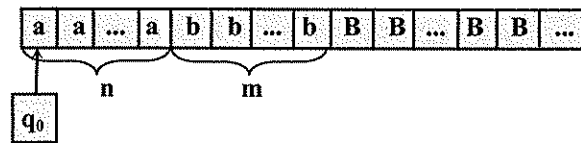
$$F = \{q_9\}$$



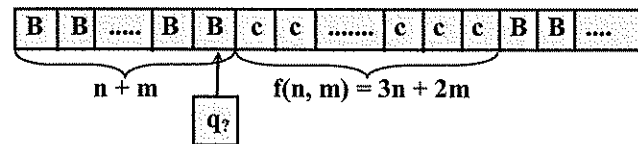
S.6.22. $f(n, m) = 3n + 2m$ $n \geq 1, m \geq 1$ işlevinin değerini hesaplayan bir Turing Makinesi ($M_{S.6.22}$) tasarlamamız isteniyor.

Aşağıdaki başlangıç ve bitiş konfigürasyonlarına uygun olarak Turing makinesinin biçimsel tanımını yapıp hareket çizeneğini oluşturunuz.

Başlangıç konfigürasyonu:



Bitiş konfigürasyonu:



S.6.23. $L_{S.6.23}$ dili aşağıdaki gibi tanımlanıyor.

$$L_{S.6.23} = \{a^n b^n c^{2n} \mid n \geq 1\}$$

$L_{S.6.23}$ dilini tanıyan bir Turing makinesi tasarlayıp tanımlamanız isteniyor.

$$M_{S.6.23} = \langle Q, \Sigma, \Gamma, \delta, q_0, B, F \rangle$$

$$Q = \{q_0, \dots\}$$

$$\Sigma = \{a, b, c\}$$

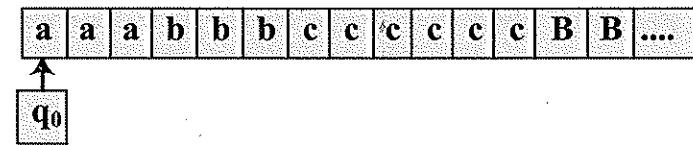
$$\Gamma = \{a, b, c, X, Y, Z, B\}$$

q_0 : Başlangıç durumu.

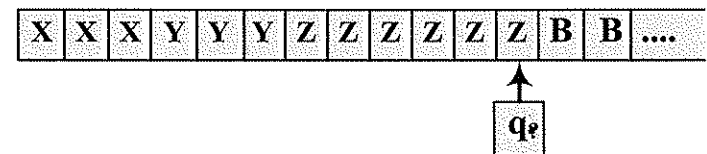
$$F = \{q_f\}$$

Dikkat : Başlangıç ve Bitiş Konfigürasyonlarının aşağıdaki gibi olması istenmektedir. Örnek dizgi $w = aabbbcccccc$

Başlangıç konfigürasyonu:



Bitiş konfigürasyonu:



Bölüm 7

Ayrıştırma İşlemleri (*Parsing*)

Ayrıştırma işleminin amacı, verilen bir tümcenin (w), belirli bir dilbilgisi (G) tarafından türetilip türetilmediğinin bulunması; eğer tümce dilbilgisi tarafından türetilmiş ise de nasıl türetildiğine ilişkin bilginin elde edilmesidir. Ayrıştırma işlemi bağlamdan-bağımsız dilbilgileri için incelenecektir. Bağlamdan-bağımsız dilbilgileri için, ayrıştırma işleminin amacının, ayrıştırma ağacının elde edilmesi olduğunu söyleyebiliriz. Eğer ayrıştırma ağacı elde edilemiyorsa, verilen tümcenin (w) ilgili dilbilgisi (G) tarafından türetilmediği; başka bir deyişle sözdizim açısından w 'nin yanlış olduğu sonucu çıkarılır. Eğer ayrıştırma ağacı elde edilir, başka bir deyişle w 'nin elde edilebilmesi için G 'nin hangi kurallarının hangi sırada uygulanması gerektiği ortaya konulursa, hem w 'nin sözdizim açısından doğru olduğu sonucu çıkarılır, hem de w 'nin taşıdığı anlam (*semantic*) belirlenmiş olur. Bu nedenle ayrıştırma işlemi derleme ya da yorumlama açısından son derece önemli bir işlemdir.

Eğer bir dilbilgisi ile, aynı tümce için birden çok soldan türetme bulunabiliyorsa bu dilbilgisi belirsizlik taşıyan (*ambiguous*) bir dilbilgisidir. Aynı tümce için birden çok soldan türetme oluşturulabilmesi, bu tümce için derleyici tarafından birden çok kod kümesi oluşturulabilmesi anlamına gelir. Bu kod kümelerinden yalnız biri tümcenin beklenen amacına karşı gelir. Diğer kod kümeleri ise, tümceyi oluşturanın düşüncesinden farklı sonuçlar doğurur.

Örnek 7.1.

$$G_{7.1} : S \Rightarrow aS \mid Sa \mid a$$
$$L(G_{7.1}) = a^* - a = a^+$$

Bu dilbilgisi belirsizlik taşıyan bir dilbilgisidir. Çünkü $w = aa$ için, birden çok soldan türetme bulunabilir.

$$S \Rightarrow aS \Rightarrow aa$$

$$S \Rightarrow Sa \Rightarrow aa$$

$G_{7.1}$ belirsizlik taşıyan bir dilbilgisidir. Ancak aşağıda tanımlanan dilbilgisi ($G'_{7.1}$) aynı dili türeten ve belirsizlik taşımayan bir dilbilgisidir.

$$G'_{7.1} : S \Rightarrow aS \mid a$$

Örnek 7.2.

$$G_{7.2} : S \Rightarrow bS \mid Sb \mid a$$

$$L(G_{7.2}) = b^* a b^*$$

Bu dilbilgisi de belirsizlik taşıyan bir dilbilgisidir. Çünkü $w = bab$ için, birden çok soldan türetme bulunabilir.

$$S \Rightarrow bS \Rightarrow bSb \Rightarrow bab$$

$$S \Rightarrow Sb \Rightarrow bSb \Rightarrow bab$$

$G_{7.2}$ belirsizlik taşıyan bir dilbilgisidir. Ancak aşağıda tanımlanan dilbilgisi ($G'_{7.2}$) aynı dili türeten ve belirsizlik taşımayan bir dilbilgisidir.

$$G'_{7.2} : S \Rightarrow bS \mid A$$

$$A \Rightarrow Ab \mid a$$

Görüldüğü gibi “belirsizlik” dilin değil dilbilgisinin bir özelliğidir. Ancak bazı diller için, dili türeten ve belirsizlik taşımayan bir dilbilgisi bulmak mümkün değildir. Bu tür dillere ise doğal olarak belirsiz (*inherently ambiguous*) diller denir.

Ayrıştırma işleminde kullanılan dilbilgilerinin, belirsizlik taşımayan, belirgin (*unambiguous*) dilbilgileri olduğunu varsayacağız.

7.1. Ayrıştırma Türleri

Ayrıştırmada iki temel yaklaşım vardır. Bu yaklaşımlar sırasıyla:

- Yukarıdan-aşağıya ayrıştırma (*top-down parsing*), ve
- Aşağıdan-yukarıya ayrıştırma (*bottom-up parsing*)

olarak adlandırılır.

7.1.1. Yukarıdan-Aşağıya Ayrıştırma (*Top-down Parsing*)

Yukarıdan-aşağıya ayrıştırmada, dilbilgisinin başlangıç simgesi olan S 'den başlanarak, ayrıştırılacak tümce (w) türetilmeye çalışılır. w türetilmeye çalışılırken soldan ya da sol öncelikli türetme (*leftmost derivation*) kuralları uygulanır. Soldan türetmede, türetmenin herhangi bir adımındaki tümcesel yapı α ise, ve α içindeki

sözdizim değişkenlerinden en soldaki A ise, A kurallarından biri uygulanarak yeni bir tümcesel yapı elde edilir:

$$S^* \Rightarrow \alpha \quad \alpha = uA\beta \quad : \quad \alpha \in V^+, \beta \in V^*, A \in V_N, u \in V_T^*$$

Salt uç simgelerden oluşan u , tümcesel yapının uç ya da terminal önekidir. u 'nın aynı zamanda ayrıştırılan sözcüğün (w) de öneki olması gerekir. Eğer u w 'nin öneki değilse, $\alpha = uA\beta$ tümcesel yapısından w türetilemez. Yukarıdan-aşağıya ayrıştırma süreci bir ağaçla gösterilirse, bu ağaç aşağıdaki gibi oluşturulur:

- Ağacın kökünün etiketi S 'dir.
- Dilbilgisinin her ($S \Rightarrow \alpha$) yeniden yazma kuralı için, ağacın 2. düzeyinde etiketi α olan bir düğüm oluşturulur.
- Ağacın herhangi bir düğümünün etiketi $uA\beta$ ise, bu düğümün altında, her ($A \Rightarrow \gamma$) yeniden yazma kuralı için, etiketi $u\gamma\beta$ olan yeni bir düğüm oluşturulur.
- Ağacın herhangi bir düğümünün etiketi $uA\beta$ ise, ve de u w 'nin öneki değilse, bu düğüm bir ölü düğümdür ve bu düğümün altında türetme yapılamaz. Eğer bir düğümün etiketi salt u ise, ve de u w 'den farklı ise bu düğüm de bir ölü düğümdür.
- Türetme işlemi ağacın bir düğümünün etiketi w oluncaya, ya da ağacın tüm yaprakları birer ölü düğüm oluncaya kadar sürdürülür. Eğer etiketi w olan bir düğüm elde edilirse ayrıştırma olumlu sonuçlanmış olur. Eğer etiketi w olan bir düğüm elde edilemez ve ağacın tüm yapraklarının birer ölü düğüm olduğu anlaşılırsa, ayrıştırma olumsuz sonuçlanmış olur.
- Eğer dilbilgisi belirgin (*unambiguous*) bir dilbilgisi ise, ağacın birden çok düğümünün etiketi aynı olamaz.

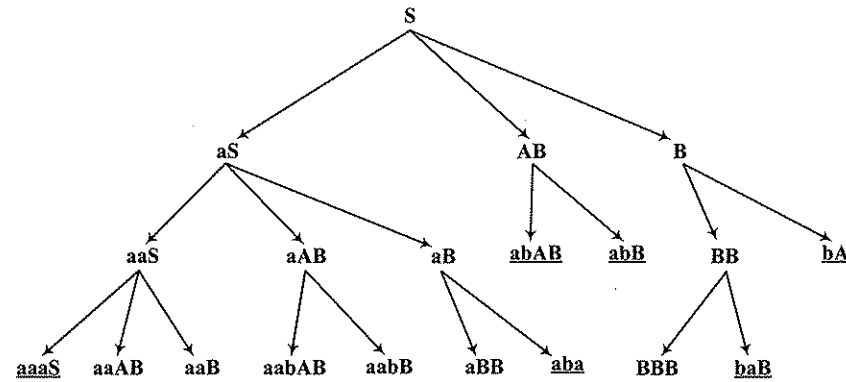
Örnek 7.3.

$$G_{7.3} : S \Rightarrow aS \mid AB \mid B$$

$$A \Rightarrow abA \mid ab$$

$$B \Rightarrow BB \mid ba$$

$w = aababbaba$ olsun. Bu tümce ayrıştırılırken kullanılabilecek ilk kural $S \Rightarrow aS$, $S \Rightarrow AB$ ya da $S \Rightarrow B$ olabilir. Dolayısıyla ağacın ikinci düzeyinde etiketleri (aS), (AB) ve (B) olan üç düğüm bulunur ve bunların hiçbirisi ölü düğüm değildir. Ağacın üçüncü düzeyinde ise yedi düğüm bulunur ve bunlardan üçü ölü düğümdür. Ağacın dördüncü düzeyinde dokuz düğüm bulunur ve bunlardan üçü ölü düğümdür. Bu ağacın ilk dört düzeyi aşağıda görülmektedir. Örnek tümce için etiketi $aababbaba$ olan düğüm ağacın sekizinci düzeyinde elde edilebilmektedir.



Örnek 7.3'te de görüldüğü gibi, yukarıdan-aşağıya ayrıştırma işlemi deterministik olmayan bir işlemdir. Ayrıştırma işleminin karmaşıklığı, dilbilgisinin yeniden yazma kurallarının sayısı ile ayrıştırılan tümcedeki simge sayısına paralel olarak üssel biçimde artar. Yukarıdan-aşağıya ayrıştırma için kullanılabilecek iki algoritma vardır. Bu algoritmalar:

- Genişlik-öncelikli yukarıdan-aşağıya ayrıştırma (*Breadth-first top-down parsing*)
- Derinlik-öncelikli yukarıdan-aşağıya ayrıştırma (*Depth-first top-down parsing*)

algoritmaları olarak adlandırılır. Her iki algoritma da deterministik değildir. Oysa uygulama açısından ayrıştırma işleminin deterministik olması son derece önemlidir. Deterministik ayrıştırma konusu bir sonraki kesimde (bkz 7.2) ele alınacaktır.

7.1.2. Aşağıdan-Yukarıya Ayrıştırma (*Bottom-up Parsing*)

Aşağıdan-yukarıya ayrıştırmada, ayrıştırılacak tümceden (w) başlanarak bir dizi işlem sonunda dilbilgisinin başlangıç simgesi olan S elde edilmeye çalışılır. Aşağıdan-yukarıya ayrıştırmada, genellikle sağdan ya da sağ öncelikli türetme (*rightmost derivation*) kullanılır. Buna göre, bir tümcenin (w) aşağıdan-yukarıya ayrıştırılmasında, bu tümcenin sağdan türetilmesinde yapılan işlemler tersine yapılır. Sağdan türetmenin herhangi bir adımındaki tümcesel yapı $\alpha A v$ ise, ve A en sağdaki değişken ise, A kurallarından birini ($A \Rightarrow \beta$) kullanarak, $\alpha \beta v$ 'nin elde edilebileceğini biliyoruz.

$$S^* \Rightarrow \alpha A v \Rightarrow \alpha \beta v \Rightarrow \dots \Rightarrow w$$

Sağdan türetmeyle w yukarıdaki gibi elde edildiğine göre, w aşağıdan-yukarıya ayrıştırılırken, $\alpha \beta v$ tümcesel yapısı elde edildiğinde, dilbilgisinin ($A \Rightarrow \beta$) kuralı kullanılarak, $\alpha A v$ elde edilir. $\alpha \beta v$ tümcesel yapısında, β yerine A konulmasına **indirgeme** (*reduction*) denir. Aşağıdan-yukarıya ayrıştırmanın amacı, w 'den başlayarak bir dizi indirgeme sonunda S 'yi elde etmektir. Eğer w 'den başlanarak S

elde edilebiliyorsa hem w 'nin sözdizim açısından doğru bir sözcük olduğu anlaşılır, hem de w 'nin sağdan türetilmesi elde edilmiş olur. Örneğin $G_{7.3}$ ile, sağ öncelikli olarak $w = aababbaba$ aşağıdaki gibi türetilir:

$$S \Rightarrow aS \Rightarrow aAB \Rightarrow aAB\bar{B} \Rightarrow aA\bar{B}ba \Rightarrow aA\bar{b}aba \Rightarrow aabA\bar{b}aba \Rightarrow aababbaba$$

Aynı sözcüğün aşağıdan yukarıya-ayrıştırılması da aşağıdaki gibi gerçekleşir:

$$aababbaba \Rightarrow aabA\bar{b}aba \Rightarrow aA\bar{b}aba \Rightarrow aAB\bar{B}a \Rightarrow aAB\bar{B} \Rightarrow aA\bar{B} \Rightarrow aS \Rightarrow S$$

Yukarıdan-aşağıya ayrıştırma işlemi gibi aşağıdan-yukarıya ayrıştırma işlemi de deterministik olmayan bir işlemdir. Aşağıdan-yukarıya ayrıştırma işleminin karmaşıklığı da, dilbilgisinin yeniden yazma kurallarının sayısı ile ayrıştırılan tümcedeki simge sayısına paralel olarak üssel biçimde artar. Aşağıdan-yukarıya ayrıştırma için de:

- Genişlik-öncelikli aşağıdan-yukarıya ayrıştırma (*Breadth-first bottom-up parsing*)
- Derinlik-öncelikli aşağıdan-yukarıya ayrıştırma (*Depth-first bottom-up parsing*)

algoritmaları kullanılabilir. Bu algoritmalar da deterministik değildir. Hangi koşullarda aşağıdan-yukarıya ayrıştırmanın deterministik bir işleme dönüştürülebileceği bir sonraki kesimde (bkz.7.2) ele alınacaktır.

7.2. Deterministik Ayrıştırma

Ayrıştırma işlemi, ancak belirli özellikleri taşıyan dilbilgileri için deterministik bir sürece dönüştürülebilir. Ayrıştırmanın deterministik bir sürece dönüştürülmesinde kullanılan ortak teknik ise **ileri-bakış** (*lookahead*) tekniğidir.

Deterministik yukarıdan-aşağıya ayrıştırmayı, ileri-bakış tekniği kullanarak gerçekleştirmeyi olanaklı kılan bağlamdan-bağımsız dilbilgileri **LL(k) Dilbilgileri** olarak adlandırılır. Buna karşılık, derterministik aşağıdan-yukarıya ayrıştırmayı, ileri-bakış tekniği kullanarak gerçekleştirmeye olanak veren dilbilgileri **LR(k) Dilbilgileri** olarak adlandırılır.

7.2.1. LL(k) Dilbilgileri

LL(k) adlandırılmasında:

- ilk L ayrıştırılacak sözcüğün soldan sağa doğru tarandığını,
- ikinci L, ayrıştırıcının soldan (sol öncelikli) türetme gerçekleştirdiğini,
- k ise ileri-bakış simgesi sayısını

gösterir. Buna göre bağlamdan-bağımsız bir dilbilgisinin **LL(k)** dilbilgisi olması, bu dilbilgisi tarafından türetilen tümcelerin, soldan-sağa doğru taranarak ve k ileri-bakış simgesi kullanılarak, yukarıdan-aşağıya türetme ile deterministik olarak ayrıştırılabileceğini gösterir.

Yukarıdan-aşağıya ayrıştırma işlemlerinde ileri-bakış tekniğinin kullanılmasını bir örnekle göstermeye çalışalım. Örnek olarak $G_{7.3}$ ile türetilen $w = aababbaba$ sözcüğünün ayrıştırılmasını düşünelim. Bu dilbilgisinin üç **S** kuralı vardır. Ayrıştırma işleminin deterministik olmasını sağlamak için, ayrıştırmanın her adımında tek bir yeniden yazma kuralı kullanılabilir. w 'nin ilk simgesi **a**'dır. **S** kurallarından ilk ikisi kullanılarak **a** ile başlayan tümceler türetilir. Ancak $(S \Rightarrow B)$ kuralı kullanılarak **a** ile başlayan tümceler türetilmez. Böylece w 'nin ilk simgesine baktığımızda kullanılabilecek kural sayısını üçten ikiye indirebiliriz. Ancak bu deterministik ayrıştırma için yeterli değildir. Oysa w 'nin ilk iki simgesine baktığımızda, **aa** ile başlayan tümcenin türetilmesi için ilk olarak $(S \Rightarrow aS)$ kuralının kullanılması gerektiği görülür. $S \Rightarrow aS$ kuralı ile w 'nin ilk simgesi türetilmiş olur. Bu aşamada sonraki iki simge **ab**'dir. **ab** simgeleri, **S** kurallarından $(S \Rightarrow aS)$ ya da $(S \Rightarrow AB)$ 'nin kullanılabileceğini gösterir; ancak bu iki kuraldan hangisinin kullanılacağına karar vermek için yetersiz kalır. Demekki, $G_{7.3}$ dilbilgisi tarafından türetilen tümceler yukarıdan-aşağıya ayrıştırılırken, ayrıştırmanın deterministik olması için iki ileri-bakış simgesi de yeterli değildir. Başka bir deyişle $G_{7.3}$ bir **LL(2)** dilbilgisi değildir. Aslında bu dilbilgisi bir **LL(k)** dilbilgisi değildir. $w = ababababab....$ örneğini düşünelim. Bu sözcüğün türetilmesinde kullanılan ilk kural $(S \Rightarrow aS)$ ya da $(S \Rightarrow AB)$ olabilir. Bu iki kuraldan hangisinin kullanıldığını belirleyebilmek için ya sözcük sonunu ya da **bb** örüntüsünü görmek gerekir.

Örnek 7.4.

$$G_{7.4} : \quad \begin{array}{l} S \Rightarrow aS \mid cA \\ A \Rightarrow bA \mid cB \mid \lambda \\ B \Rightarrow cB \mid a \mid \lambda \end{array}$$

Bu dilbilgisi için yukarıdan-aşağıya deterministik ayrıştırma bir ileri-bakış simgesi yeterlidir. Eğer değişken **S** ise ileri-bakış simgesinin **a** ya da **c** olmasına göre $(S \Rightarrow aS)$ ya da $(S \Rightarrow cA)$ kuralının kullanıldığı anlaşılır. Buna karşılık eğer değişken **A** ise ileri-bakış simgesinin **b** ya da **c** olmasına göre $(A \Rightarrow bA)$ ya da $(A \Rightarrow cB)$ kuralının kullanıldığı anlaşılır. Eğer değişken **B** ise ileri-bakış simgesinin **c** ya da **a** olmasına göre $(B \Rightarrow cB)$ ya da $(B \Rightarrow a)$ kuralının uygulandığı anlaşılır. Değişken **A** ya da **B** iken dizgi sonuna (*end of string*) ulaşılmışsa da $(A \Rightarrow \lambda)$ ya da $(B \Rightarrow \lambda)$ kuralının uygulandığı anlaşılır. Sonuç olarak $G_{7.4}$ dilbilgisi bir **LL(1)** dilbilgisidir ve bu dilbilgisi tarafından türetilen tümcelerin yukarıdan-aşağıya ayrıştırılması, bir ileri-bakış simgesi ile deterministik biçimde yürütülebilir. Örnek olarak $G_{7.4}$ tarafından türetilmiş ($w = acbb$) sözcüğünü düşünelim. Bu sözcüğün yukarıdan-aşağıya ayrıştırılması aşağıdaki gibi özetlenebilir.

Türetilmiş olan uç örnek	İleri-bakış simgesi	Uygulanan kural	Türetilen tümcesel yapı
-	a	$S \Rightarrow aS$	aS
a	c	$S \Rightarrow cA$	acA
ac	b	$A \Rightarrow bA$	acbA
acb	b	$A \Rightarrow bA$	acbbA
acbb	EoS	$S \Rightarrow \lambda$	acbb

7.2.2. LL(k) Ayrıştırma

LL(k) dilbilgileri tarafından türetilen diller için kullanılabilen deterministik ayrıştırma **LL(k)** ayrıştırma olarak adlandırılabilir. **LL(k)** ayrıştırma işleminde PDA modeli ile birlikte ayrıştırma çizelgesi olarak adlandırılan bir çizelge kullanılır. Ayrıştırma kullanılan PDA modeli ile ayrıştırma çizelgesinin nasıl oluşturulacağı aşağıda açıklanmaktadır.

7.2.2.1. LL(k) Ayrıştırma Kullanılacak PDA Modeli

LL(k) dilbilgisi $G = \langle V_N, V_T, P, S \rangle$ olsun.

Bu dilbilgisi tarafından türetilen dilin ayrıştırılmasında 3 durumlu bir PDA kullanılır. Yukarıdan-aşağıya ayrıştırma algoritmasına uygun biçimde oluşturulan bu PDA kısaca **PDA(↓)** olarak nitelenecektir.

$$M = \langle Q, \Sigma, \Gamma, \delta, q_0, Z_0, \Phi \rangle$$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = V_T$$

$$\Gamma = V_N \cup V_T \cup \{Z_0\}$$

$$F = \{q_2\}$$

$$\delta : \quad 1) \quad \delta(q_0, \lambda, \lambda) = (q_1, S)$$

$$2) \quad \forall a \in V_T : \quad \delta(q_1, a, a) = (q_1, \lambda)$$

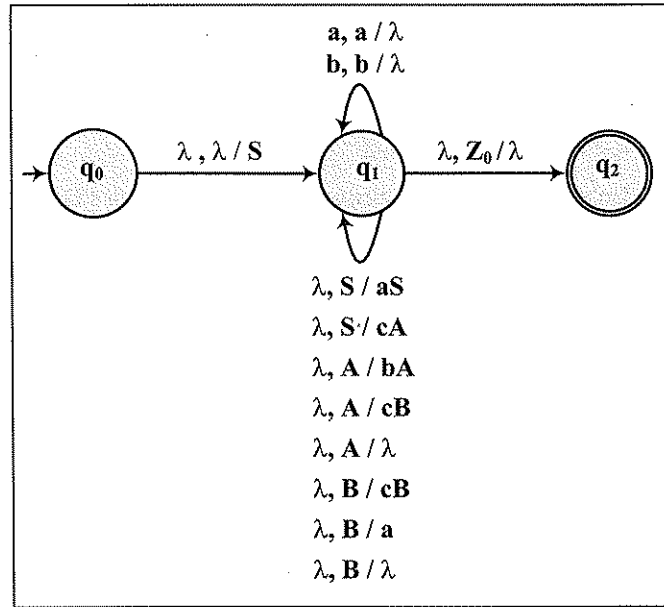
$$\forall (A \Rightarrow \beta) \in P : \quad \delta(q_1, \lambda, A) = (q_1, \beta)$$

$$3) \quad \delta(q_1, \lambda, Z_0) = (q_2, \lambda)$$

Tanımlanan PDA hem boş yığıtla hem de uç durumla tanıyan bir PDA'dır. Doğru bir sözcük için hem yığıt boşalmakta, hem de sonlu denetim birimi bir uç duruma (q_2) ulaşmaktadır. **PDA(↓)**'nin çalışması aşağıdaki gibi açıklanabilir:

- Başlangıçta PDA q_0 durumundadır ve yığıtta Z_0 vardır.
- İlk olarak bir λ -hareketi ile yığıtta S eklenir, ve PDA q_1 durumuna geçer.
- q_1 durumunda iken, yığıtın üstünde bir uç simge (a) varsa ve giriş dizgisinden okunan simge de aynı simge (a) ise, yığıttan bu simge silinir ve giriş dizgisinden bir sonraki simge okunur.
- q_1 durumunda iken, yığıtın üstünde bir değişken (A) varsa, ayrıştırma çizelgesi yardımıyla bu değişkene ilişkin kurallardan biri seçilir ve bu kuralın sağ tarafı (β) λ -hareketi ile yığıtta eklenir.
- q_1 durumunda iken, yığıtın üstünde Z_0 varsa, λ -hareketi ile Z_0 yığıttan silinir.
- Sonuçta giriş dizgisinin sonuna ulaşıldığında, yığıt boşalmışsa ve sonlu denetim birimi q_2 durumundaysa sözcük tanınır.

Örnek 7.4'deki $G_{7.4}$ dilbilgisi için LL(1) ayrıştırmada kullanılacak PDA($\downarrow$) modeli Çizim 7.1'de görülmektedir.



Çizim 7.1. $G_{7.4}$ 'ün LL(1) Ayrıştırılması için PDA($\downarrow$) Modeli

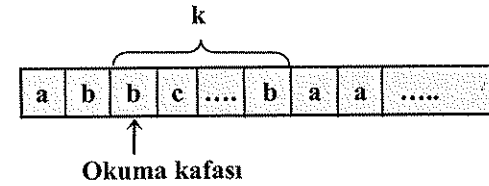
7.2.2.2. LL(k) Ayrıştırma Çizelgesinin Oluşturulması

Ayrıştırma çizelgesi, ayrıştırmadaki belirsizliği ortadan kaldırmak ve ayrıştırmanın deterministik bir biçimde sürdürülmesini sağlamak için kullanılan bir çizelgedir. Yukarıda sunulan PDA modeli incelendiğinde, yığıtın üstünde bir değişken bulunduğu belirsizliğin ortaya çıktığı görülür. Ayrıştırma çizelgesi, ileri-bakış

simgelerine bakılarak, ilgili değişkenle ilgili yeniden yazma kurallarından hangisinin kullanılacağını belirler ve böylece ayrıştırmadaki belirsizliği ortadan kaldırır. Ayrıştırma çizelgesinde:

- Dilbilgisindeki her değişken için bir satır,
- İleri-bakış simgelerinin her birleşimi için de bir kolon

bulunur. Çizelgenin her elemanında ise hangi yeniden yazma kuralının kullanılacağı, ya da sözcüğün yanlış olduğunun anlaşılacağı gösterilir. Örnek bir LL(k) dilbilgisi için ayrıştırma çizelgesinin nasıl oluşturulacağını göstermeden önce ileri-bakış simgelerinin anlamını netleştirmekte yarar vardır. Ayrıştırma işlemi bir PDA modeli ile gerçekleştirildiği için, Çizim 5.1'de yer alan PDA'nın soyut makine modeli geçerli olacaktır. Bu modele göre, giriş dizgisi bir şerit üzerinde kayıtlıdır ve belirli bir anda okuma kafası şeridin bir hücresi üzerinde bulunmaktadır. İleri-bakış simgelerinin hangileri olduğu aşağıdaki çizimde görülmektedir.



Yukarıdaki çizimde de görüldüğü gibi, ileri-bakış simgeleri, okuma kafasının üzerinde bulunduğu simgeden başlayan k simgedir. Başka bir deyişle o an okunan simge ileri-bakış simgelerine dahildir ve k 'nın değeri en az 1'dir. Bazı kaynaklarda ileri-bakış simgeleri, o an okunan simgenin sağındaki k simge olarak tanımlanır. Bu tanıma göre k 'nın değeri 0 olabilir. Bu kitapta, ileri-bakış simgeleri, o an okunan simge dahil k simge olarak tanımlanmaktadır.

Ayrıştırma çizelgesinde, ileri-bakış simgelerinin her birleşimi için bir kolon bulunacağı belirtildi. İleri-bakış simgelerinin birleşimleri konusuna da açıklık getirmek gerekir. Okuma kafasının üzerinde bulunduğu simge, dilbilgisinin bir uç simgesi olabileceği gibi, dizgi sonu (ds) da olabilir. Dolayısıyla dizgi sonunu da bir simge olarak düşünmek gerekir. Buna göre:

- uç simgeleri a, b ve c olan bir LL(1) dilbilgisinin ayrıştırma çizelgesinde, başlıkları a, b, c ve ds olan 4 kolon bulunacaktır.
- uç simgeleri a, b ve c olan bir LL(2) dilbilgisinin ayrıştırma çizelgesinde, başlıkları $aa, ab, ac, ba, bb, bc, ca, cb, cc, ads, bds, cds$ ve ds olan 13 kolon bulunacaktır.
- Uç simgeleri a, b ve c olan bir LL(3) dilbilgisinin ayrıştırma çizelgesinde ise, başlıkları $aaa, aab, aac, aba, abb, abc, aca, acb, acc, baa, bab, bac, bba, bbb, bbc, bca, bcb, bcc, caa, cab, cac, cba, cbb, cbc, cca, ccb, ccc, aads, abds, acds, bads, bbds, bcds, cads, cbds, ccds, ads, bds, cds$ ve ds olan 40 kolon bulunacaktır.

Çizelge 7.1'de, Örnek 7.4'deki dilbilgisi için oluşturulan ayrıştırma çizelgesi görülmektedir. $G_{7.4}$ bir LL(1) dilbilgisi olduğu için çizelgede sadece 4 kolon vardır. Çizelgenin birçok elemanında yer alan "yan" ifadesi, bu koşullarda sözcüğün yanlış olduğunun anlaşılacağını göstermektedir. Örneğin değişken S, ileri-bakış simgesi ise b ise sözcüğün yanlış olduğu anlaşılır. Çünkü S ile başladığında, b ile başlayan bir alt dizgi üretilmez.

Çizelge 7.1. $G_{7.4}$ için LL(1) Ayrıştırma Çizelgesi

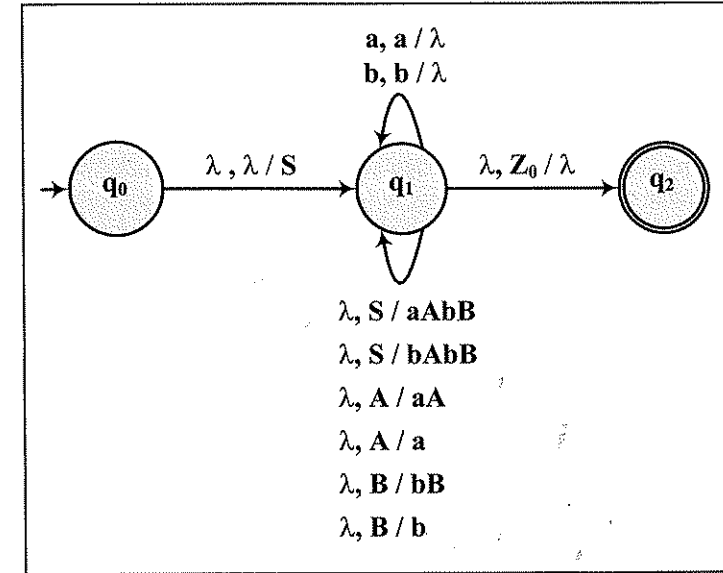
	a	b	c	ds
S	$S \Rightarrow aS$	yan	$S \Rightarrow cA$	yan
A	yan	$A \Rightarrow bA$	$A \Rightarrow cB$	$S \Rightarrow \lambda$
B	$B \Rightarrow a$	yan	$B \Rightarrow cB$	$S \Rightarrow \lambda$

Örnek 7.5.

$$G_{7.5} : \begin{array}{l} S \Rightarrow aAbB \mid bAbB \\ A \Rightarrow aA \mid a \\ B \Rightarrow bB \mid b \end{array}$$

Bu dilbilgisinin türettiği dilin tümcelerinin yukarıdan-aşağıya deterministik biçimde ayrıştırılabilmesi için bir ileri-bakış simgesi yeterli değildir. Ancak iki ileri-bakış simgesi kullanıldığında yukarıdan-aşağıya deterministik ayrıştırma yapılması mümkündür. Başka bir deyişle bu dilbilgisi bir LL(2) dilbilgisidir.

$G_{7.5}$ 'in yukarıdan-aşağıya deterministik ayrıştırması için kullanılacak PDA'nın geçiş çizeneği Çizim 7.2'de, ayrıştırma çizelgesi ise Çizelge 7.2'de görülmektedir.



Çizim 7.2. $G_{7.5}$ 'in LL(2) Ayrıştırılması için PDA($\downarrow$) Modeli

Çizelge 7.2. $G_{7.5}$ için LL(2) Ayrıştırma Çizelgesi

	aa	ab	ba	bb	ads	bds	ds
S	$S \Rightarrow aAbB$	yan	$S \Rightarrow bAbB$	yan	yan	yan	yan
A	$A \Rightarrow aA$	$A \Rightarrow a$	yan	yan	yan	yan	yan
B	yan	yan	yan	$B \Rightarrow bB$	yan	$B \Rightarrow b$	yan

7.2.3. LR(k) Dilbilgileri

LR(k) adlandırmasında:

- L ayrıştırılacak sözcüğün soldan sağa doğru tarandığını,
- R ayrıştırıcının sağdan (sağ-öncelikli) türetme gerçekleştirdiğini,
- k ise ileri-bakış simgesi sayısını

gösterir.

Bilindiği gibi aşağıdan-yukarıya ayrıştırma, ayrıştırılacak sözcük (w) üzerinde bir dizi indirgeme (*reduction*) yapılarak dilbilgisinin başlangıç simgesi elde edilmeye çalışılır. Bunun için başlangıçta $w = uv$ olarak bölünür ve örüntü eşleme (*pattern matching*) tekniği ile, sağ tarafı u 'nun soneki olan bir yeniden yazma kuralı bulunmaya çalışılır. Ancak bu işlem deterministik olmayan ve belirsizlik düzeyi yüksek bir işlemdir. Çünkü öncelikle w 'nin uv biçiminde bölünmesi tek değildir. Ayrıca sağ tarafı u 'nun son ekine eşit birden çok yeniden yazma kuralı bulunabilir. Bu nedenle aşağıdan-yukarıya ayrıştırma algoritması deterministik olmayan bir algoritmadır. LR(k) ayrıştırma ise, k ileri-bakış simgesi ile aşağıdan-yukarıya ayrıştırmanın deterministik bir biçimde yürütüldüğü bir ayrıştırma algoritmasıdır.

Aşağıdan-yukarıya ayrıştırma algoritması, yukarıdan-aşağıya ayrıştırma algoritmasına göre daha karmaşık ve belirsizlik düzeyi daha yüksek bir algoritmadır. Buna paralel olarak LR(k) ayrıştırma da, LL(k) ayrıştırmaya göre daha karmaşık bir ayrıştırma algoritmasıdır. Bu nedenle eğer bir dilbilgisi hem LL(k), hem de LR(k) dilbilgisi ise, bu dilbilgisi için LL(k) ayrıştırma tercih edilir. LR(k) olan, ancak LL(k) olmayan dilbilgileri için ise, deterministik ayrıştırma yapılmak istendiğinde LR(k) ayrıştırma tek seçenektir.

7.2.4. LR(k) Ayrıştırma

LR(k) dilbilgileri tarafından türetilen diller için kullanılabilen deterministik ayrıştırma LR(k) ayrıştırma olarak adlandırılabilir. LR(k) ayrıştırma işleminde PDA modeli ile birlikte ayrıştırma çizelgesi olarak adlandırılan bir çizelge kullanılır. Ayrıştırmada kullanılan PDA modeli ile ayrıştırma çizelgesinin nasıl oluşturulacağı aşağıda açıklanmaktadır.

7.2.4.1. LR(k) Ayrıştırmada Kullanılacak PDA Modeli

LR(k) dilbilgisi $G = \langle V_N, V_T, P, S \rangle$ olsun.

Bu dilbilgisi tarafından türetilen dilin ayrıştırılmasında 3 durumlu bir PDA kullanılır. Aşağıdan-yukarıya ayrıştırma algoritmasına uygun biçimde oluşturulan bu PDA kısaca $PDA(\uparrow)$ olarak nitelenecektir.

$$M = \langle Q, \Sigma, \Gamma, \delta, q_0, Z_0, \Phi \rangle$$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = V_T$$

$$\Gamma = V_N \cup V_T \cup \{Z_0\}$$

$$F = \{q_2\}$$

$$\delta: 1) \forall a \in V_T : \delta(q_0, a, \lambda) = (q_0, a)$$

$$2) \forall (A \Rightarrow \beta) \in P : \delta(q_0, \lambda, \beta) = (q_0, A)$$

$$3) : \delta(q_0, \lambda, S) = (q_1, \lambda) \\ \delta(q_1, \lambda, Z_0) = (q_2, \lambda)$$

Tanımlanan PDA hem boş yığıtla hem de uç durumla tanıyan bir PDA'dır. Doğru bir sözcük için hem yığıt boşalmakta, hem de sonlu denetim birimi bir uç duruma (q_2) ulaşmaktadır. $PDA(\uparrow)$ 'nin çalışması aşağıdaki gibi açıklanabilir:

- Başlangıçta PDA q_0 durumundadır ve yığıtta Z_0 vardır.
- PDA q_0 durumunda iken, şeritten okunan her bir uç simge (a) yığıta eklenir. Bu işleme kaydırma işlemi (*shift operation*) denilir.
- PDA q_0 durumunda iken, yığıtın üstündeki simgeler (β) bir yeniden yazma kuralının ($A \Rightarrow \beta$) sağ tarafına eşit ise, yığıttan bu simgeler (β) silinir ve yığıta A eklenir. Bu işleme de indirgeme işlemi (*reduction operation*) denilir.
- PDA q_0 durumunda iken, yığıtın üstünde S varsa, λ -hareketi ile yığıttan S silinir ve sonlu denetim birimi q_1 durumuna geçer.
- PDA q_1 durumunda iken, yığıtın üstünde Z_0 varsa, λ -hareketi ile yığıttan Z_0 silinir ve sonlu denetim birimi q_2 durumuna geçer.
- Sonuçta giriş dizgisinin sonuna ulaşıldığında, yığıt boşalmışsa ve sonlu denetim birimi q_2 durumundaysa sözcük tanınır.

Aşağıdan-yukarıya ayrıştırma işlemi yukarıda sunulan PDA modeline göre gerçekleştirildiğinde:

- kaydırma (*shift*) işlemi mi yoksa indirgeme (*reduction*) işlemi mi yapılacağı,
- eğer indirgeme işlemi yapılacaksa da, olası indirgeme işlemlerinden hangisinin yapılacağı

konusunda belirsizlik vardır. Diğer güçlük ise yığıtın üstündeki n simgenin sorgulanması ve bu simgelerin yeniden yazma kurallarından birinin sağ tarafına eşit olduğunun belirlenmesidir. Bütün bu belirsizlik ve güçlükleri aşarak ayrıştırma işlemini deterministik biçimde sürdürebilmek için ayrıştırma çizelgesi kullanılır.

Yukarıda sunulan PDA modeli, 5. Bölümde sunulan PDA modelinin genişletilmiş biçimidir. Bu modelde bir hareket, genel olarak aşağıdaki gibi gösterilmektedir:

$$\delta(q_i, s, \beta) = (q_j, \alpha) \quad s \in V_T \cup \{\lambda\} \quad \alpha, \beta \in \Gamma^*$$

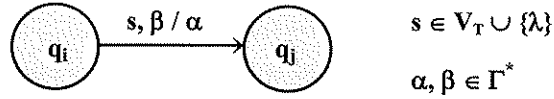
Bu gösterimde q_i ve q_j durumları dışında 3 parametre bulunmaktadır.

s : bir uç simge ya da λ . Eğer s bir uç simge ise, hareketin yapılabilmesi için giriş şeridinden okunan simgenin s olması gerekir ve hareketin yapılması ile şeritten bir sonraki simge okunur.

β : yığıt alfabesi simgelerinden oluşan bir altdizgi ya da λ . Eğer β bir altdizgi ise, hareketin yapılabilmesi için yığıtın üstünde bu altdizginin bulunması gerekir ve hareketin yapılması ile yığıttan bu altdizgi silinir.

α : yığıt alfabesi simgelerinden oluşan bir altdizgi ya da λ . Eğer α bir altdizgi ise, hareketin yapılması ile yığıtta α eklenir. Eğer α boş dizgi (λ) ise, hareketin yapılması ile yığıtta hiçbir şey eklenmez.

Bu üç parametre *read*, *pop* ve *push* parametreleri olarak anılır.

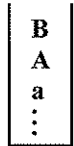


Bu üç parametreden biri ya da ikisi λ olabilir. (*read*, *s*), (*pop*, β) ve (*push*, α)'nın anlamları aşağıdaki gibidir:

read, s: hareketin yapılabilmesi için şeritten okunan son simgenin *s* olması gerekir ve hareketin yapılması ile şeritten bir sonraki simge okunur. $s = \lambda$ ise şeritten son okunan simge ne olursa olsun hareket yapılır ve hareketin yapılması ile şeritten bir sonraki simge okunmaz.

pop, β : hareketin yapılabilmesi için yığıtın üstünde β 'nin bulunması gerekir ve hareketin yapılması ile yığıtın üstünden β silinir. $\beta = \lambda$ ise yığıtın üstünde ne olursa olsun hareket yapılır ve hareketin yapılması ile yığıttan hiçbir şey silinmez.

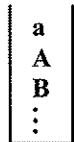
Eğer β birden çok simgeden oluşuyorsa, yığıtın üstünde β aranırken, β 'nin en sağdaki simgesi yığıtın en üstünde aranmalıdır. Örneğin $\beta = aAB$ ise, ve yığıtın en üstündeki üç simge aşağıdaki gibiyse:



yığıtın üstünde β vardır.

push, α : hareketin yapılması ile yığıtta α eklenir. $\alpha = \lambda$ ise hareketin yapılması ile yığıtta hiçbir şey eklenmez.

Eğer α birden çok simgeden oluşuyorsa, α yığıtta eklendiğinde, α 'nın en soldaki simgesi en üste gelir. Örneğin $\alpha = aAB$ ise, α 'nın eklenmesinden sonra yığıtın görünümü aşağıdaki gibi olur.



7.2.4.2. İşaretli Kurallar

Bağlamdan-bağımsız her dilbilgisi LR(k) dilbilgisi değildir. Dolayısıyla bağlamdan-bağımsız her dilbilgisi için bir LR(k) ayrıştırıcı bulmak mümkün değildir. LR(k) ayrıştırıcılar ile çözümlenebilen bağlamdan-bağımsız dilbilgileri, türettikleri diller deterministik PDA'lar tarafından tanımlanabilen dilbilgileridir. Başka bir deyişle eğer bağlamdan-bağımsız bir dilbilgisinin türettiği dil deterministik bir PDA tarafından tanımlanabiliyorsa, bu dilbilgisi bir LR(k) dilbilgisidir ve bu dilbilgisinin türettiği tümceler bir LR(k) ayrıştırıcı tarafından ayrıştırılabilir.

LR(k) ayrıştırma çizelgesinin oluşturulması sonlu özdevinir (FA) modelinin önemli uygulama alanlarından biridir. Bu kitapta yalnız LR(1) ayrıştırma çizelgesinin nasıl oluşturulacağına yer verilecektir.

LR(1) ayrıştırma çizelgesinin nasıl oluşturulacağına geçmeden önce, birkaç kavramın tanımlanmasına gerek vardır. Tanımlanacak kavramlar “işaretli kural” ve “işaretli kural kümesinin kapanışı” kavramlarıdır.

İşaretli Kural (Marked Rule)

Bağlamdan-bağımsız bir dilbilgisinin yeniden yazma kurallarının $A \Rightarrow \beta$ biçiminde olduğunu biliyoruz. Bu gösterimde β uç simge ve sözdizim değişkenlerinden oluşan bir dizgidir. Bu dizginin herhangi bir yerine “.” işareti konularak oluşturulan kurala işaretli kural denir. Örneğin ($S \Rightarrow aAb$) yeniden yazma kuralını düşünelim. Bu kuraldan 4 işaretli kural oluşturulabilir :

$$S \Rightarrow .aAb \quad S \Rightarrow a.Ab \quad S \Rightarrow aA.b \quad S \Rightarrow aAb.$$

Bu işaretli kurallardan ilki “başlangıç biçimini”, sonraki iki tanesi “ara biçimleri”, sonuncusu ise “sonuç biçimini” oluşturmaktadır.

İşaretli Kural Kümesinin Kapanışı (Closure of a set of Marked Rule)

K belirli sayıda işaretli kuraldan oluşan bir küme olsun. Bu kümenin kapanışını (K^+) elde etmek için, kümedeki kurallardan, “.”dan hemen sonra bir değişkenin (A) yer aldığı kuralar belirlenir ve bu değişkenlere ilişkin işaretli kuralların ilk biçimleri kümeye eklenir. Yeni eklenen kurallarda “.”dan hemen sonra bir değişken (A) yer alıyorsa, aynı işlemler, kümeye yeni kural eklenmesi bitene kadar tekrarlanır.

Örneğin bir dilbilgisinin yeniden yazma kuralları:

$$\begin{array}{l|l} S \Rightarrow aS & A \\ A \Rightarrow Ab & B \\ B \Rightarrow Bc & C \\ C \Rightarrow Cc & c \end{array}$$

olsun. Dilbilginin 8 yeniden yazma kuralınının 20 biçimi vardır. Bunlardan 8'i başlangıç biçimi, 8'i sonuç biçimi, 4'ü de ara biçimdir. Bu dilbilgisi için,

$$K = \{ S \Rightarrow a.S \\ C \Rightarrow .c \}$$

işaretili kural kümesinin kapanışı aşağıdaki gibi bulunur:

$$K^+ = \{ S \Rightarrow a.S \\ C \Rightarrow .c \\ S \Rightarrow .aS \\ S \Rightarrow .A \\ A \Rightarrow .Ab \\ A \Rightarrow .B \\ B \Rightarrow .Bc \\ B \Rightarrow .C \\ C \Rightarrow .Cc \\ C \Rightarrow .c \}$$

7.2.4.3. LR(1) Ayırıştırıcı Geçiş Çizeneğinin Oluşturulması

Bir LR(1) dilbilgisi verildiğinde, bu dilbilgisi için LR(1) ayırıştırıcının geçiş çizeneği aşağıdaki gibi oluşturulur.

- Dilbilgisine yeni bir değişken (S') ve yeni bir kural ($S' \Rightarrow S$) eklenir. Bu kuralın işaretli başlangıç biçimi olan ($S' \Rightarrow .S$)'nin kapanışı geçiş çizeneğinin 1 numaralı düğümünü oluşturur.
- Geçiş çizeneğindeki her düğüm için, bu düğümde yer alan işaretli kurallarda “.” nın sağında yer alan ($A \Rightarrow \dots X \dots$) her simgeye (X : uç simge ya da sözdizim değişkeni) karşılık bir geçiş oluşturulur. X simgesi ile başlayan geçişin ucundaki düğümde, “.” nın X 'in sağına geçmesi ile elde edilen ($A \Rightarrow \dots X \dots$) işaretli kuralların kapanışı yer alacaktır. Eğer elde edilecek işaretli kurallar kümesinin kapanışı mevcut düğümlerden birinde yer alıyorsa, X geçişi bu düğüme yapılır; yoksa geçiş çizeneğine yeni bir düğüm eklenir, düğümün içine ilgili işaretli kurallar kümesinin kapanışı yazılır ve X geçişi bu yeni eklenen düğümde son bulur.
- Eğer bir düğümdeki işaretli kural bir sonuç biçimi ($A \Rightarrow \dots X$) ise, bu kural düğümdeki tek kural olmalıdır. Bu durumda bu düğüm bir uç düğüm olur ve çift çember ile gösterilir.
- Geçiş çizeneği, çizeneğe eklenen her yeni düğümden başlayan geçişler tamamlanuncaya kadar sürdürülür.

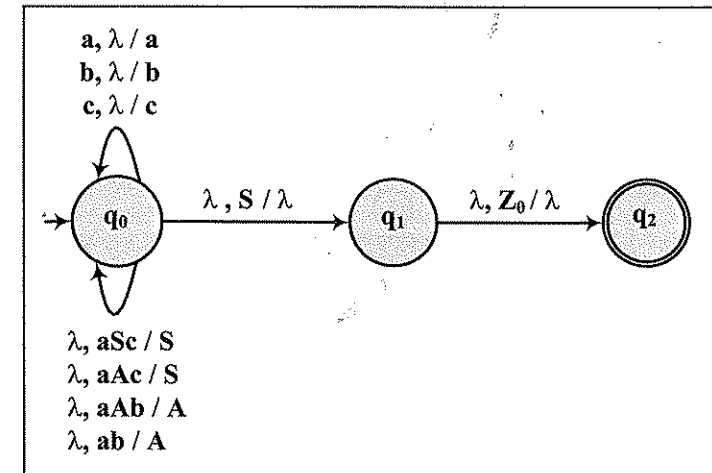
Örnek 7.6.

$$G_{7.6} : S \Rightarrow aSc \mid aAc \\ A \Rightarrow aAb \mid ab$$

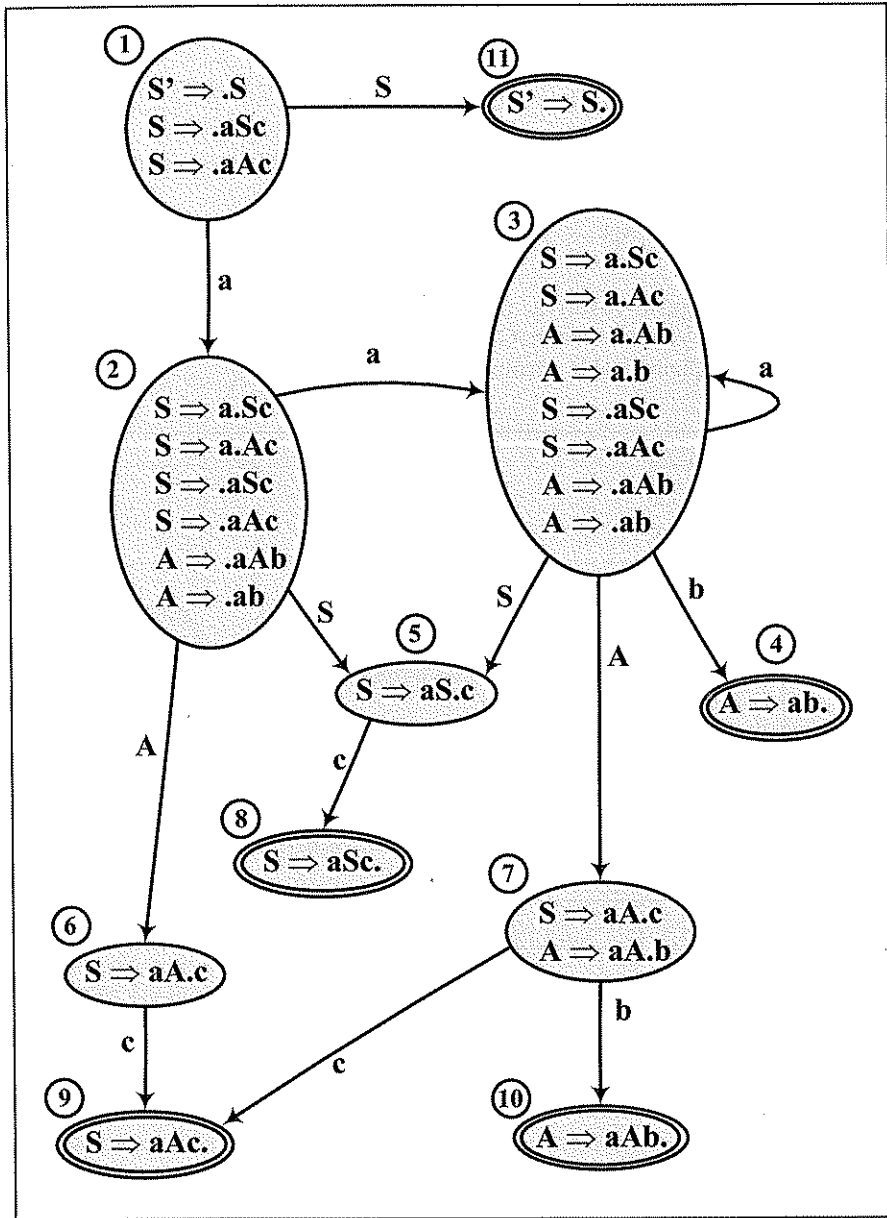
$G_{7.6}$ bağlamdan-bağımsız bir dilbilgisidir. Bu dilbilgisinin türettiği dili aşağıdaki gibi tanımlayabiliriz.

$$L(G_{7.6}) = \{ a^n b^m c^k \mid m, k \geq 1, n = m + k \}$$

$G_{7.6}$ 'nın aşağıdan-yukarıya ayrıştırılmasında kullanılacak PDA($\uparrow$) modeli Çizim 7.3'de yer almaktadır. $G_{7.6}$ için oluşturulacak LR(1) ayırıştırıcının geçiş çizeneği ise Çizim 7.4'de görülmektedir.



Çizim 7.3. $G_{7.6}$ 'nın LR(1) Ayırıştırılması için PDA($\uparrow$) Modeli

Çizim 7.4. $G_{7.6}$ İçin LR(1) Ayrıştırıcı Geçiş Çizeneği

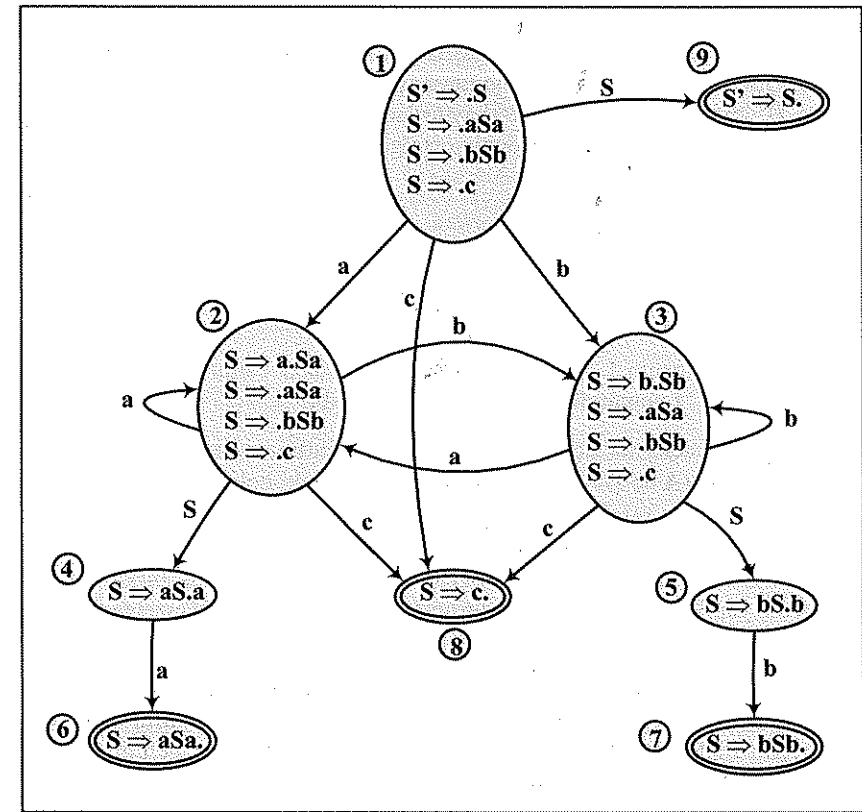
Örnek 7.7.

$$G_{7.7} : S \Rightarrow aSa \mid bSb \mid c$$

$G_{7.7}$ bağlamdan-bağımsız bir dilbilgisidir. Dilbilgisinin türettiği dili aşağıdaki gibi tanımlayabiliriz.

$$L(G_{7.7}) = \{ucu^R \mid u \in (a+b)^*\}$$

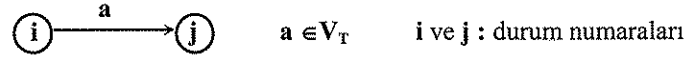
$G_{7.7}$ için oluşturulacak LR(1) ayrıştırıcının geçiş çizeneği çizim 7.5'de görülmektedir.

Çizim 7.5. $G_{7.7}$ İçin LR(1) Ayrıştırıcı Geçiş Çizeneği

7.2.4.4. LR(1) Ayırıştırma Çizelgesinin Oluşturulması

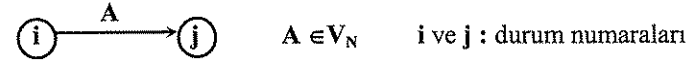
LR(1) ayırıştırıcı geçiş çizeneğindeki bilgilerden yararlanılarak, LR(1) ayırıştırma çizelgesi aşağıdaki gibi oluşturulur.

- Geçiş çizeneğindeki sonlu özdevinirin her durumu için çizelgede bir satır bulunur.
- Dilbilgisindeki her uç simge ve değişken ile dizgi sonu (**ds**) için çizelgede bir kolon bulunur.
- Geçiş çizeneğindeki her



geçiş için, çizelgenin *i*. satır *a* başlıklı kolonundaki elemanına “**Kaydır, j**” yazılır. Bu ifadenin anlamı “*a*’yı yığta kaydır (*shift a*) ve *j* numaralı duruma git (*go to j*)” dir. Yığta eklenecek (kaydırılacak) simgenin hangisi olduğu kolon başlığında görülmektedir. Bu nedenle çizelgede “**Kaydır, j**” kısa gösterimine yer verilmektedir.

- Geçiş çizelgesindeki her



geçiş için, çizelgenin *i*. satır *A* başlıklı kolonundaki elemanına “**j**” yazılır. Bu ifadenin anlamı “*j* numaralı duruma git (*go to j*)” dir.

- Geçiş çizeneğindeki her uç durum için, eğer uç durumun numarası *i*, durumdaki işaretli kural da ($A \Rightarrow \beta$) ise, çizelgenin *i*. satırındaki belirli hücelere “**İndirge $\beta \Rightarrow A$** ” yazılır. *i*. satırda, hangi kolonlardaki hücelere bu ifadenin yazılacağı, $\beta \Rightarrow A$ indirgemesi yapıldığında, bir sonraki simgenin ne olacağına göre belirlenir (bir sonraki simge uç simgeler ile **ds**’den biri ya da birkaçı olabilir). İndirgeme yapıldıktan sonra hangi duruma gidileceği ise, geçiş çizelgesinde, ilgili uç durumdan geriye doğru iz sürülerek bulunur.
- Geçiş çizeneğinde ($S' \Rightarrow S$) işaretli kuralının bulunduğu durumun numarası *k* ise, ayırıştırma çizelgesinin *k*. satırında **ds** başlıklı hücreye “**kabul et (accept)**” yazılır. Bu ifadenin anlamı, ayırıştırma sürecinde bu noktaya gelinirse dizginin geçerli bir tümce olduğunun anlaşılacağı ve kabul edileceğidir.

$G_{7.6}$ ve $G_{7.7}$ dilbilgileri için oluşturulan LR(1) Ayırıştırma Çizelgeleri sırasıyla Çizelge 7.3 ve Çizelge 7.4’de görülmektedir.

Çizelge 7.3. $G_{7.6}$ İçin LR(1) Ayırıştırma Çizelgesi

Dur.	a	b	c	S	A	ds
1	Kaydır, 2	-	-	11	-	-
2	Kaydır, 3	-	-	5	6	-
3	Kaydır, 3	Kaydır, 4	-	5	7	-
4	-	İndirge ($ab \Rightarrow A$)	İndirge ($ab \Rightarrow A$)	-	-	-
5	-	-	Kaydır, 8	-	-	-
6	-	-	Kaydır, 9	-	-	-
7	-	Kaydır, 10	Kaydır, 9	-	-	-
8	-	-	İndirge ($aSc \Rightarrow S$)	-	-	İndirge ($aSc \Rightarrow S$)
9	-	-	İndirge ($aAc \Rightarrow S$)	-	-	İndirge ($aAc \Rightarrow S$)
10	-	İndirge ($aAb \Rightarrow A$)	İndirge ($aAb \Rightarrow A$)	-	-	-
11	-	-	-	-	-	Kabul et

Çizelge 7.4. $G_{7.7}$ İçin LR(1) Ayırıştırma Çizelgesi

Drm.	a	b	c	S	ds
1	Kaydır, 2	Kaydır, 3	Kaydır, 8	9	-
2	Kaydır, 2	Kaydır, 3	Kaydır, 8	4	-
3	Kaydır, 2	Kaydır, 3	Kaydır, 8	5	-
4	Kaydır, 6	-	-	-	-
5	-	Kaydır, 7	-	-	-
6	İndirge ($aSa \Rightarrow S$)	İndirge ($aSa \Rightarrow S$)	-	-	İndirge ($aSa \Rightarrow S$)
7	İndirge ($bSb \Rightarrow S$)	İndirge ($bSb \Rightarrow S$)	-	-	İndirge ($bSb \Rightarrow S$)
8	İndirge ($c \Rightarrow S$)	İndirge ($c \Rightarrow S$)	-	-	İndirge ($c \Rightarrow S$)
9	-	-	-	-	Kabul

7.2.4.5. LR(1) Ayırıştırma Sürecinin Yürütülmesi

LR(1) ayırıştırma işlemi, aşağıdan-yukarıya ayırtırmaya uygun olarak oluşturulan $PDA(\uparrow)$ modeli ile LR(1) ayırıştırma çizelgesi kullanılarak gerçekleştirilir. LR(1) ayırıştırma çizelgesi ile, $PDA(\uparrow)$ modelinin taşıdığı belirsizlik (kaydırma mı yoksa indirgeme mi yapılacak? İndirgeme yapılırsa hangi kurala göre yapılacak?) çözülür ve ayırıştırma işlemi deterministik bir işleme dönüştürülür. Her kaydırma işleminden sonra hangi duruma gidileceği ayırıştırma çizelgesinde yer almaktadır. Her indirgeme işleminden sonra hangi duruma gidileceği ise ayırıştırma çizelgesinde yer almaz. Bunun ayırıştırma sırasında dinamik biçimde bulunması gerekir. Bunun için de, kaydırma işlemlerinde, yığta bir simge eklenirken, kaydırma işleminin hangi durumda yapıldığı bilgisinin de yığta eklenmesi gerekir. Ayırıştırma sürecinin nasıl yürütüldüğünü ve her indirgeme işleminden sonra hangi duruma gidileceğinin nasıl bulunduğu bir örnek üzerinde açıklanacaktır.

Örnek olarak $G_{7,6}$ dilbilgisi tarafından türetilmiş olan

$$w = aaabcc$$

tümcesi için ayırıştırma sürecinin nasıl yürütüleceği incelenecektir. Ayırıştırma işlemlerinde Çizim 7.3'deki $PDA(\uparrow)$ modeli ile Çizelge 7.3'de yer alan ayırıştırma çizelgesi kullanılır.

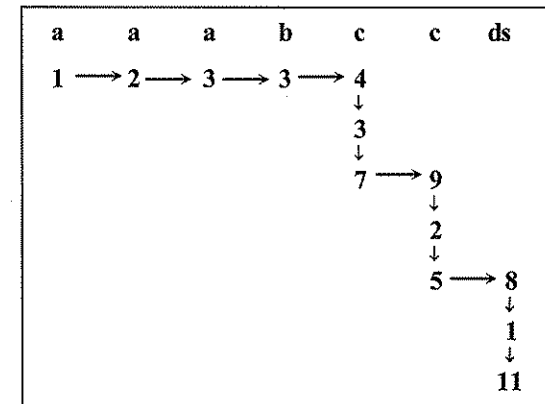
1. Başlangıçta PDA q_0 durumundadır, yığta Z_0 simgesi vardır, okuma kafası w 'nin ilk simgesi (a) üzerindedir. Ayırıştırıcı ise 1 numaralı durumdadır (ayırıştırma çizelgesinin 1. satırı).
2. İlk işlem olarak w 'nin ilk simgesi (a) yığta kaydırılır. Kaydırma işlemi 1 numaralı durumda yapıldığı için, yığta a simgesinin yanına durum numarası olarak da 1 yazılır. Kaydırma işleminden sonra ayırıştırıcı 2 numaralı duruma gider.
3. İkinci yapılan işlem w 'nin ikinci simgesinin (a) yığta kaydırılmasıdır. Kaydırma işlemi 2 numaralı durumda yapıldığı için, yığta a simgesinin yanına durum numarası olarak 2 yazılır. Kaydırma işleminden sonra ayırıştırıcı 3 numaralı duruma gider.
4. Üçüncü işlem olarak w 'nin üçüncü simgesi (a) yığta kaydırılır. Kaydırma işlemi 3 numaralı durumda yapıldığı için, yığta a simgesinin yanına durum numarası olarak 3 yazılır. Kaydırma işleminden sonra ayırıştırıcı 3 numaralı durumda kalır.
5. Dördüncü işlem olarak w 'nin dördüncü simgesi (b) yığta kaydırılır. Kaydırma işlemi 3 numaralı durumda yapıldığı için, yığta b simgesinin yanına durum numarası olarak da 3 yazılır. Kaydırma işleminden sonra ayırıştırıcı 4 numaralı duruma gider.
6. Ayırıştırıcı 4 numaralı durumda bulunduğu, ileri bakış simgesi de w 'nin beşinci simgesi (c) olduğu için, beşinci olarak yapılacak işlem ($ab \Rightarrow A$) indirgeme işlemidir. Bunun için yığtadan ab çıkarılır ve yığta A eklenir. Yığtadan ab çıkarıldığında 3 numaralı duruma dönmüş olur. Çünkü ab 'nin ilk simgesi olan a yığta 3 numaralı durumda eklenmiştir. Ayırıştırma çizelgesine göre 3 numaralı durumda A işlendiği için de ayırıştırıcı 7 numaralı duruma geçer.

7. İndirgeme işleminde giriş şeridi değişmediği için ileri bakış simgesi (w 'nin beşinci simgesi olan c) de değişmemiştir. Ayırıştırıcı da 7 numaralı durumda bulunduğu için altıncı işlem olarak w 'nin beşinci simgesi (c) yığta kaydırılır. Kaydırma işlemi 7 numaralı durumda yapıldığı için, yığta c simgesinin yanına durum numarası olarak da 7 yazılır. Kaydırma işleminden sonra ayırıştırıcı 9 numaralı duruma gider.
8. Ayırıştırıcı 9 numaralı durumda bulunduğu, ileri bakış simgesi de w 'nin son simgesi (c) olduğu için, yedinci olarak yapılacak işlem ($aAc \Rightarrow S$) indirgeme işlemidir. Bunun için yığtadan aAc çıkarılır ve yığta S eklenir. Yığtadan aAc çıkarıldığında 2 numaralı duruma dönmüş olur. Çünkü aSc 'nin ilk simgesi olan a yığta 2 numaralı durumda eklenmiştir. Ayırıştırma çizelgesine göre 2 numaralı durumda S işlendiği için de ayırıştırıcı 5 numaralı duruma geçer.
9. İndirgeme işleminde giriş şeridi değişmediği için ileri bakış simgesi (w 'nin son simgesi olan c) de değişmemiştir. Ayırıştırıcı da 5 numaralı durumda bulunduğu için sekizinci işlem olarak w 'nin son simgesi (c) yığta kaydırılır. Kaydırma işlemi 5 numaralı durumda yapıldığı için, yığta c simgesinin yanına durum numarası olarak da 5 yazılır. Kaydırma işleminden sonra ayırıştırıcı 8 numaralı duruma gider.
10. Ayırıştırıcı 8 numaralı durumda bulunduğu, ileri bakış simgesi de ds (dizgi sonu) olduğu için, dokuzuncu olarak yapılacak işlem ($aSc \Rightarrow S$) indirgeme işlemidir. Bunun için yığtadan aSc çıkarılır ve yığta S eklenir. Yığtadan aSc çıkarıldığında 1 numaralı duruma dönmüş olur. Çünkü aSc 'nin ilk simgesi olan a yığta 1 numaralı durumda eklenmiştir. Ayırıştırma çizelgesine göre 1 numaralı durumda S işlendiği için de ayırıştırıcı 11 numaralı duruma geçer.
11. Ayırıştırıcı 11 numaralı durumda bulunduğu, ileri bakış simgesi de ds olduğu için ayırıştırma süreci biter ve ($w = aaabcc$) tümcesi kabul edilir.

Yukarıda ayrıntılı olarak anlatılan ayırıştırma sürecini, aşağıdaki çizimle özetlemek mümkündür.

Dizgi (w) :

Durumlar dizisi :



Sorular

S.7.1. Bağlamdan-bağımsız (CF) $G_{S.7.1}$ dilbilgisi aşağıdaki gibi tanımlanıyor.

$$G_{S.7.1} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S\}$$

$$V_T = \{a, b, c, d\}$$

$$P: S \Rightarrow bSc \mid bSd \mid a$$

- Bu dilbilgisi bir $LL(k)$ dilbilgisi midir? Niçin? Evet ise: $\{a, b, c, d\}$ alfabesinde bir tümce (w) verildiğinde, bu tümcenin $G_{S.7.1}$ tarafından türetilmiş bir tümce olup olmadığını anlamak için yapılacak ayrıştırmada kaç ileri-bakış simgesi (*lookahead symbols*) kullanmak gerekir?
- $G_{S.7.1}$ 'in $LR(1)$ dilbilgisi olduğu biliniyor. Aşağıdan-yukarıya ayrıştırmada $L(G_{S.7.1})$ dilini tanıyacak $PDA(\uparrow)$ 'nin tanımını oluşturunuz.
- $G_{S.7.1}$ dilbilgisi için $LR(1)$ ayrıştırma çizelgesini (*parsing table*) oluşturunuz.

S.7.2. $L_{S.7.2}$ dili, $\{a, b, c\}$ alfabesinde aşağıda tanımlanan dilbilgisi tarafından türetilmektedir.

$$G_{S.7.2} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, X, Y\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow aS \mid aX \mid \lambda$$

$$X \Rightarrow bS \mid bX \mid bY$$

$$Y \Rightarrow cS \mid cY$$

- Bu dilbilgisi bir $LL(k)$ dilbilgisi midir? Evet ise: $\{a, b, c\}$ alfabesinde bir tümce (w) verildiğinde, bu tümcenin $G_{S.7.2}$ tarafından türetilmiş bir tümce olup olmadığını anlamak için yapılacak ayrıştırmada kaç ileri-bakış simgesi (*lookahead symbols*) kullanmak gerekir?
- $LL(k)$ ayrıştırmada kullanılacak $PDA(\downarrow)$ 'nin tanımını oluşturunuz.
- $LL(k)$ ayrıştırma çizelgesini oluşturunuz.

S.7.3. Bağlamdan-bağımsız (CF) $G_{S.7.3}$ dilbilgisi aşağıdaki gibi tanımlanıyor:

$$G_{S.7.3} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow A$$

$$A \Rightarrow aAa \mid aAb \mid ab$$

- Aşağıdan-yukarıya ayrıştırmada $L(G_{S.7.3})$ dilini tanıyacak $PDA(\uparrow)$ 'nin tanımını oluşturunuz.
- $G_{S.7.3}$ dilbilgisi için $LR(1)$ ayrıştırma çizelgesini (*parsing table*) oluşturunuz.
- Oluşturduğunuz ayrıştırma çizelgesine göre, $w = aaabab$ tümcesi ayrıştırılırken hangi durumlardan geçileceğini bulunuz.
($1 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow \dots 6 \rightarrow 7$ gibi)

S.7.4. $G_{S.7.4}$ dilbilgisinin yeniden yazma kuralları aşağıdaki gibidir.

$$P: S \rightarrow aAbB \mid bAbB$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bB \mid b$$

Bu dilbilgisi bir $LL(k)$ dilbilgisi midir? Evet ise:

- $\{a, b\}$ alfabesinde bir tümce (w) verildiğinde, bu tümcenin $G_{S.7.4}$ tarafından türetilmiş bir tümce olup olmadığını anlamak için yapılacak ayrıştırmada kaç ileri-bakış simgesi (*lookahead symbols*) kullanmak gerekir?
- Yukarıdan-aşağıya ayrıştırmada kullanılacak $PDA(\downarrow)$ 'nin tanımı ile $LL(k)$ ayrıştırma çizelgesini oluşturunuz.

S.7.5. Bağlamdan-bağımsız (CF) $G_{S.7.5}$ dilbilgisi aşağıdaki gibi tanımlanıyor:

$$G_{S.7.5} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A\}$$

$$V_T = \{a, b, c, d\}$$

$$P: S \Rightarrow aSb \mid aAb$$

$$A \Rightarrow cAd \mid cd$$

- Bu dilbilgisinin türettiği dili tanımlayan matematiksel bir gösterim oluşturunuz.
- Aşağıdan-yukarıya ayrıştırma $L(G_{S.7.5})$ dilini tanıyacak $PDA(\uparrow)$ 'nin tanımını oluşturunuz.
- $G_{S.7.5}$ dilbilgisi için **LR(1)** ayrıştırma çizelgesini (*parsing table*) oluşturunuz.
- Oluşturduğunuz ayrıştırma çizelgesine göre, $w = aaccddbb$ tümcesi ayrıştırılırken hangi durumlardan geçileceğini bulunuz.

S.7.6. Bağlamdan-bağımsız (CF) $G_{S.7.6}$ dilbilgisi aşağıdaki gibi tanımlanıyor:

$$G_{S.7.6} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S\}$$

$$V_T = \{0, 1\}$$

$$P : S \Rightarrow 0S10 \mid 010$$

- Aşağıdan-yukarıya ayrıştırma $L(G_{S.7.6})$ dilini tanıyacak $PDA(\uparrow)$ 'nin tanımını oluşturunuz.
- $G_{S.7.6}$ dilbilgisi için **LR(1)** ayrıştırma çizelgesini (*parsing table*) oluşturunuz.
- Oluşturduğunuz ayrıştırma çizelgesine göre, $w = 000101010$ tümcesi ayrıştırılırken hangi durumlardan geçileceğini bulunuz.

S.7.7. Bağlamdan-bağımsız (CF) $G_{S.7.7}$ dilbilgisi aşağıdaki gibi tanımlanıyor:

$$G_{S.7.7} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P : S \Rightarrow ABA$$

$$A \Rightarrow aA \mid a$$

$$B \Rightarrow bBa \mid ba$$

- Bu dilbilgisi bir **LL(k)** dilbilgisi midir? Evet ise ayrıştırma kaç ileri-bakış simgesi (*lookahead symbols*) kullanmak gerekir?
- $L(G_{S.7.7})$ 'nin küme tanımını yazınız.
- Yukarıdan-aşağıya ayrıştırma $L(G_{S.7.7})$ 'yi tanıyacak $PDA(\downarrow)$ 'nin tanımını oluşturunuz.
- $G_{S.7.7}$ için **LL(k)** ayrıştırma çizelgesini oluşturunuz.

S.7.8. $G_{S.7.8}$ dilbilgisi aşağıdaki gibi tanımlanıyor:

$$G_{S.7.8} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, X, Y\}$$

$$V_T = \{a, b\}$$

$$P : S \Rightarrow aSa \mid aXa$$

$$X \Rightarrow bXb \mid bYb$$

$$Y \Rightarrow aYa \mid aba$$

- Yukarıda tanımlanan dilbilgisinin ($G_{S.7.8}$) türettiği dilin küme tanımını oluşturunuz.
- Bu dilbilgisi bir **LL(k)** dilbilgisi midir? Evet ise: $\{a, b\}$ alfabesinde bir tümce (w) verildiğinde, bu tümcenin $G_{S.7.8}$ tarafından türetilmiş bir tümce olup olmadığını anlamak için yapılacak çözümlemede kaç ileri-bakış simgesi (*lookahead symbols*) kullanmak gerekir?
- Ayrıştırma kullanılacak $PDA(\downarrow)$ 'nin geçiş çizeneği ile, ayrıştırmanın deterministik olmasını sağlayacak **LL(k)** ayrıştırma çizelgesini (*parsing table*) oluşturunuz.

S.7.9. Bağlamdan-bağımsız (CF) $G_{S.7.9}$ dilbilgisi aşağıdaki gibi tanımlanıyor:

$$G_{S.7.9} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P : S \Rightarrow Ab$$

$$A \Rightarrow AB \mid B$$

$$B \Rightarrow aAb \mid ab$$

- Bu dilbilgisinin bir **LR(1)** dilbilgisi olup olmadığını anlamak için **LR(1)** ayrıştırıcı geçiş çizeneğini oluşturunuz. Bu geçiş çizeneğine göre $G_{S.7.9}$ **LR(1)** dilbilgisi midir? Niçin?
- $L(G_{S.7.9})$ dilini tanıyan $PDA(\uparrow)$ 'nin tanımını oluşturunuz.
- $G_{S.7.9}$ dilbilgisi için **LR(1)** ayrıştırma çizelgesini (*parsing table*) oluşturunuz.

S.7.10. $G_{S.7.10} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow aA \mid bB$$

$$A \Rightarrow abA \mid baA \mid bbB \mid b$$

$$B \Rightarrow baB \mid aaA \mid abB \mid a$$

$G_{S.7.10}$ dilbilgisinin türü nedir?

$G_{S.7.10}$ bir $LL(k)$ dilbilgisi midir? Evet ise k 'nın değerini bulunuz ve $LL(k)$ ayrıştırma çizelgesini oluşturunuz.

S.7.11. $\{a, b, c\}$ alfabesinde tanımlı $L_{S.7.11}$ dili aşağıda yeniden yazma kuralları verilen $G_{S.7.11}$ dilbilgisi tarafından türetilmektedir.

$$P: S \Rightarrow aaSbb \mid aaAbb$$

$$A \Rightarrow c$$

$G_{S.7.11}$ bir $LR(1)$ dilbilgisidir. $G_{S.7.11}$ için $LR(1)$ ayrıştırıcı geçiş çizeneğini oluşturunuz.

S.7.12. Bağlamdan bağımsız (CF) $G_{S.7.12}$ dilbilgisi aşağıdaki gibi tanımlanıyor:

$$G_{S.7.12} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow aAa \mid bBb \mid a \mid b$$

$$A \Rightarrow aAa \mid b$$

$$B \Rightarrow bBb \mid a$$

Bu dilbilgisi bir $LL(k)$ dilbilgisi midir? Niçin? Evet ise k 'nın değerini bulunuz ve $LL(k)$ ayrıştırma çizelgesini oluşturunuz.

S.7.13. Bağlamdan bağımsız (CF) $G_{S.7.13}$ dilbilgisi aşağıdaki gibi tanımlanıyor:

$$G_{S.7.13} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow aaSab \mid aaab$$

a) Bu dilbilgisi bir $LL(k)$ dilbilgisi midir?

Evet ise k 'nın değeri kaçtır?

Hayır ise niçin?

b) $G_{S.7.13}$ 'ün $LR(1)$ dilbilgisi olduğu biliniyor. $G_{S.7.13}$ için $LR(1)$ ayrıştırıcı geçiş çizeneğini oluşturunuz.

S.7.14. $\{0, 1\}$ alfabesinde $G_{S.7.14}$ dilbilgisi aşağıdaki yeniden yazma kuralları ile tanımlanıyor.

$$S \Rightarrow 1A \mid 10B \mid 10$$

$$A \Rightarrow 1B \mid 1 \mid 10B \mid 10$$

$$B \Rightarrow 1B \mid 1 \mid 10B \mid 10$$

$G_{S.7.14}$ bir $LL(k)$ dilbilgisi midir?

▪ Hayır ise örnek bir dizgi ile bunu kanıtlayınız.

▪ Evet ise $LL(k)$ ayrıştırma çizelgesini oluşturunuz (ayrıştırma çizelgesinde tüm elemanları "yanlış" olan kolonlara yer vermeyiniz).

S.7.15. Bağlamdan bağımsız (CF) $G_{S.7.15}$ dilbilgisi aşağıdaki gibi tanımlanıyor:

$$G_{S.7.15} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b, c, d\}$$

$$P: S \Rightarrow AB$$

$$A \Rightarrow aAb \mid ab$$

$$B \Rightarrow cBd \mid cd$$

$LR(1)$ dilbilgisi olan $G_{S.7.15}$ için ayrıştırıcı geçiş çizeneğini oluşturunuz.

S.7.16. $G_{S.7.16}$ dilbilgisi aşağıdaki gibi tanımlanıyor:

$$G_{S.7.16} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, X, Y\}$$

$$V_T = \{a, b, c\}$$

$$P : S \Rightarrow aSa \mid aXa$$

$$X \Rightarrow bXa \mid bYa$$

$$Y \Rightarrow cYa \mid c$$

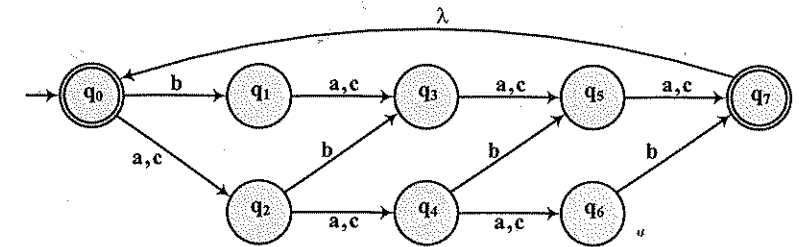
- Tanımı verilen dilbilgisinin ($G_{S.7.16}$) türettiği dilin küme tanımını oluşturunuz.
- Bu dilbilgisi bir $LL(k)$ dilbilgisi midir? Evet ise k 'nın değerini belirtip $LL(k)$ ayrıştırma çizelgesini (*parsing table*) oluşturunuz.

Seçilmiş Soruların Yanıtları

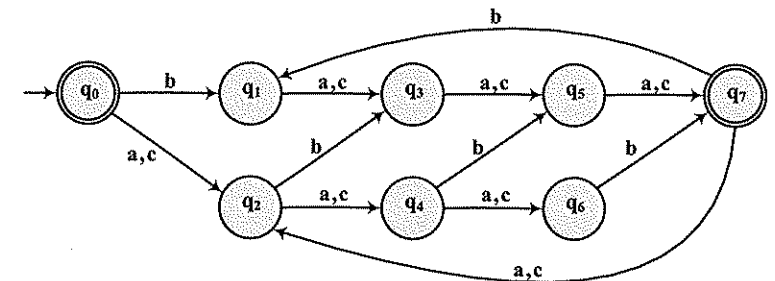
Bölüm 1

Y.1.1.

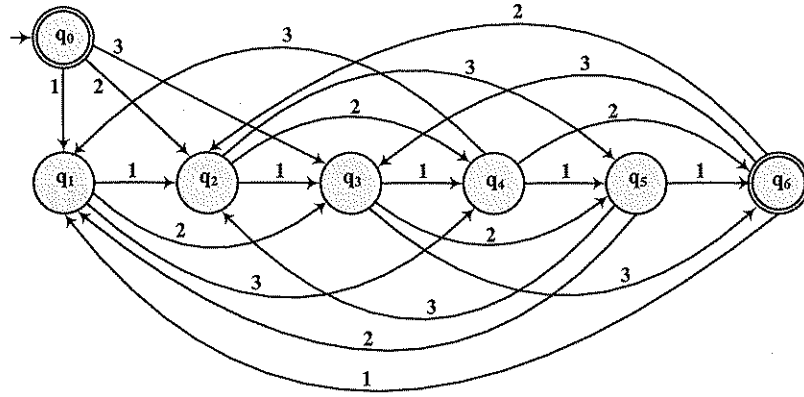
a)



λ -geçişsiz denk Geçiş Çizeneği:



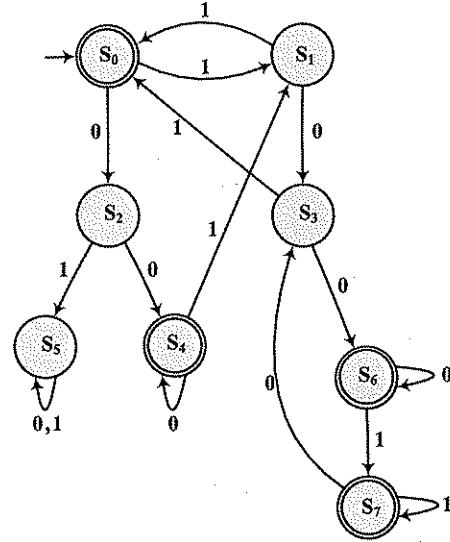
b)



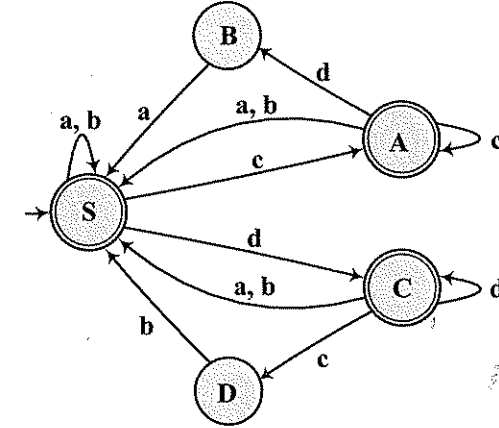
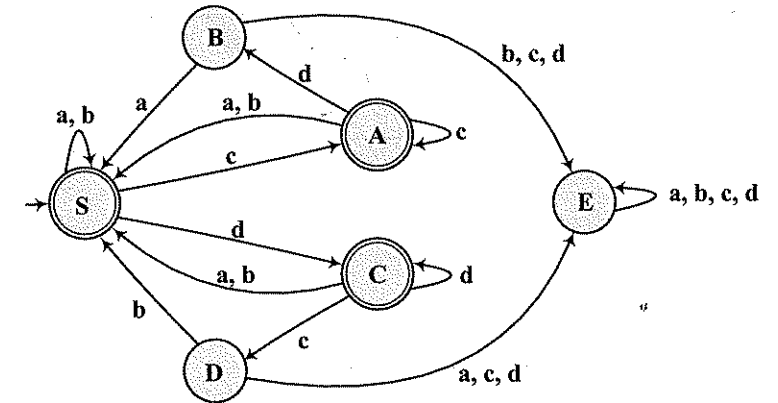
Y.1.2. Eşdeğer DFA'nın
Geçiş Çizelgesi:

	0	1
→ S ₀	S ₂	S ₁
S ₁	S ₃	S ₀
S ₂	S ₄	S ₅
S ₃	S ₆	S ₀
S ₄	S ₄	S ₁
S ₅	S ₅	S ₅
S ₆	S ₆	S ₇
S ₇	S ₃	S ₇

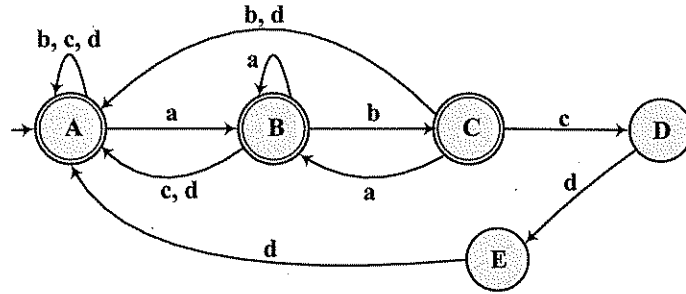
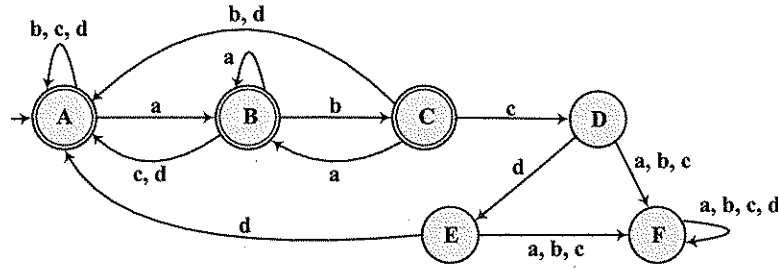
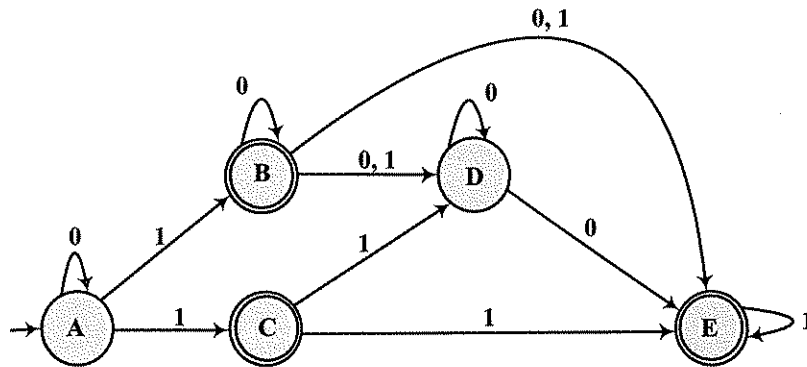
Geçiş Çizeneği:



Y.1.3.

a) $L_{S.1.3}$ 'ü tanıyan NFA'nın Geçiş Çizeneği:b) $L_{S.1.3}$ 'ü tanıyan DFA'nın Geçiş Çizeneği:

Y.1.4.

a) $L_{S,1,4}$ 'ü tanıyan NFA'nın Geçiş Çizeneği:b) $L_{S,1,4}$ 'ü tanıyan DFA'nın Geçiş Çizeneği:Y.1.7. λ -geçişsiz denk Geçiş Çizeneği:

Geçiş Çizelgesi:

	0	1
→ A	A	BC
(B)	BDE	DE
(C)	-	DE
D	DE	-
(E)	-	E

Y.1.8.

a) NFA'nın Geçiş Çizelgesi:

	0	1
→ A	AB	-
B	C	AB
(C)	BD	C
D	DE	C
E	D	AE

Başlangıç Durumunun Ardılları:

	0	1
→ A	AB	Φ
AB	ABC	AB
Φ	Φ	Φ
(ABC)	ABCD	ABC
(ABCD)	ABCDE	ABC
(ABCDE)	ABCDE	ABCE
(ABCE)	ABCD	ABCE

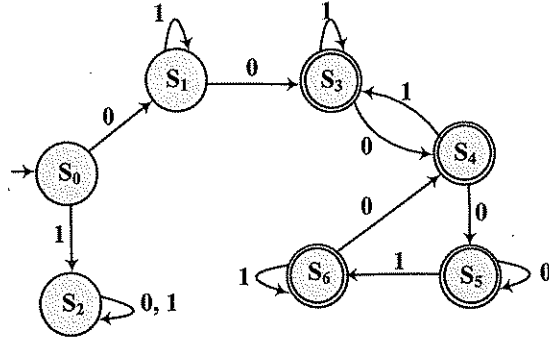
Durumların yeniden adlandırılması:

- A için : S_0
- AB için : S_1
- Φ için : S_2
- ABC için : S_3
- ABCD için : S_4
- ABCDE için : S_5
- ABCE için : S_6

DFA'nın Geçiş Çizelgesi:

	0	1
→ S_0	S_1	S_2
S_1	S_3	S_1
S_2	S_2	S_2
(S_3)	S_4	S_3
(S_4)	S_5	S_3
(S_5)	S_5	S_6
(S_6)	S_4	S_6

DFA'nın Geçiş Çizeneği:



Y.1.9.

NFA'nın Geçiş Çizelgesi:

	a	b
→A	ABE	D
B	B	AC
C	E	D
D	E	-
E	E	-

Başlangıç Durumunun Ardılları:

	a	b
→A	ABE	D
ABE	ABE	ACD
D	E	Φ
ACD	ABE	D
E	E	Φ
Φ	Φ	Φ

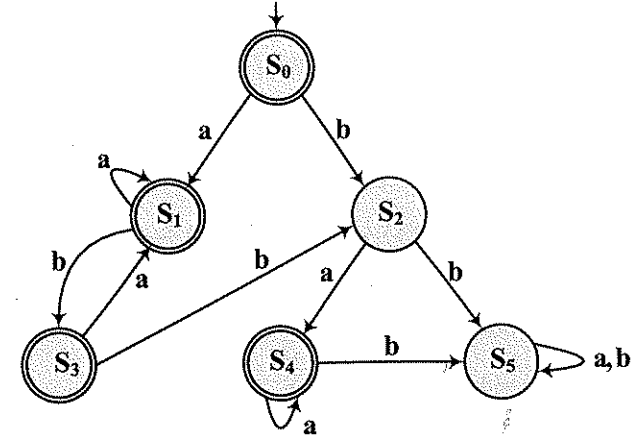
Durumların yeniden adlandırılması:

A için : S_0
 ABE için : S_1
 D için : S_2
 ACD için : S_3
 E için : S_4
 Φ için : S_5

DFA'nın Geçiş Çizelgesi:

	a	b
→S ₀	S ₁	S ₂
S ₁	S ₁	S ₃
S ₂	S ₄	S ₅
S ₃	S ₁	S ₂
S ₄	S ₄	S ₅
S ₅	S ₅	S ₅

DFA'nın Geçiş Çizeneği:



Y.1.11.

$$a) w_1 = 11010 : (q_0, 11010) \vdash (1, q_2, 1010) \vdash (11, q_3, 010) \vdash (110, q_1, 10) \vdash (11, q_2, 010) \vdash (110, q_2, 10) \vdash (1101, q_3, 0) \vdash (11010, q_1)$$

q_1 bir uç durum olmadığı için w_1 2DFA tarafından tanınmaz.

$$b) w_2 = 100101 : (q_0, 100101) \vdash (1, q_2, 00101) \vdash (10, q_2, 0101) \vdash (100, q_2, 101) \vdash (1001, q_3, 01) \vdash (10010, q_1, 1) \vdash (1001, q_2, 01) \vdash (10010, q_2, 1) \vdash (100101, q_3)$$

w_2 2DFA tarafından tanınır.

$$c) w_3 = 011010 : (q_0, 011010) \vdash (0, q_1, 11010) \vdash (q_2, 011010) \vdash (0, q_2, 11010) \vdash (01, q_3, 1010) \vdash (0, q_2, 11010) \dots$$

ID dizisinde döngü olduğu için w_3 2DFA tarafından tanınmaz.

Y.1.12.

$$a) w_1 = 10011 : (q_0, 10011) \vdash (1, q_1, 0011) \vdash (10, q_1, 011) \vdash (100, q_1, 11) \vdash (1001, q_2, 1) \vdash (100, q_3, 11) \vdash (10, q_3, 011) \vdash (1, q_4, 0011) \vdash (10, q_0, 011) \vdash (100, q_0, 11) \vdash (1001, q_1, 1) \vdash (10011, q_2)$$

w_1 2DFA tarafından tanınır.

- b) $w_2 = 01011$: $(q_0, 01011) \vdash (0, q_0, 1011) \vdash (01, q_1, 011) \vdash$
 $(010, q_1, 11) \vdash (0101, q_2, 1) \vdash (010, q_3, 11) \vdash$
 $(01, q_3, 011) \vdash (0, q_4, 1011) \vdash (q_4, 01011) \vdash$
 $(0, q_0, 1011) \dots$

ID dizisinde döngü olduğu için w_2 2DFA tarafından tanınmaz.

- c) $w_3 = 11011$: $(q_0, 11011) \vdash (1, q_1, 1011) \vdash (11, q_2, 011) \vdash$
 $(110, q_2, 11) \vdash (11, q_3, 011) \vdash (1, q_4, 1011) \vdash$
 $(q_4, 11011)$

Okuma kafası şeridin ilk hücresinde iken sola gidemez; w_3 2DFA tarafından tanınmaz.

- d) $w_4 = 10101$: $(q_0, 10101) \vdash (1, q_1, 0101) \vdash (10, q_1, 101) \vdash$
 $(101, q_2, 01) \vdash (1010, q_2, 1) \vdash (101, q_3, 01) \vdash$
 $(10, q_4, 101) \vdash (1, q_4, 0101) \vdash (10, q_0, 101) \vdash$
 $(101, q_1, 01) \vdash (1010, q_1, 1) \vdash (10101, q_2)$

w_4 2DFA tarafından tanınır.

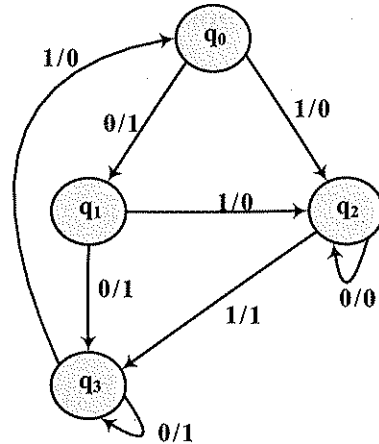
Y.1.13.

Eşdeğer Mealy makinesinin:

Durum Çizelgesi

SD	SD, z	
	x = 0	x = 1
q_0	$q_1, 1$	$q_2, 0$
q_1	$q_3, 1$	$q_2, 0$
q_2	$q_2, 0$	$q_3, 1$
q_3	$q_3, 1$	$q_0, 0$

Durum Çizeneği:



Y.1.14.

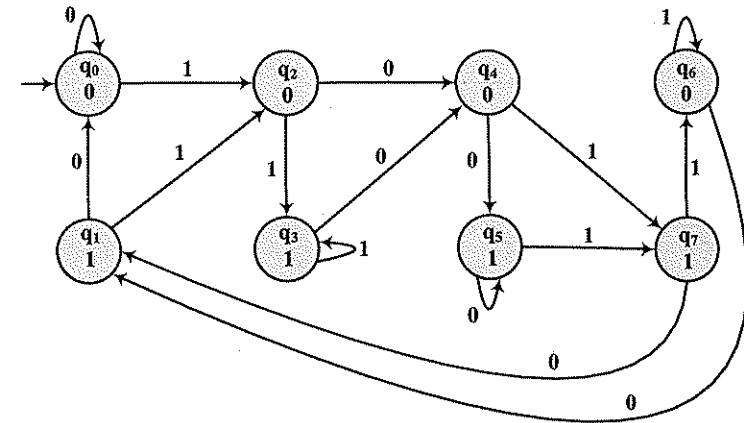
Durumların anlamları:

q_0 : [A, 0]	q_1 : [A, 1]
q_2 : [B, 0]	q_3 : [B, 1]
q_4 : [C, 0]	q_5 : [C, 1]
q_6 : [D, 0]	q_7 : [D, 1]

Moore makinesinin Durum Çizelgesi:

	0	1	z
$\rightarrow q_0$	q_0	q_2	0
q_1	q_0	q_2	1
q_2	q_4	q_3	0
q_3	q_4	q_3	1
q_4	q_5	q_7	0
q_5	q_5	q_7	1
q_6	q_1	q_6	0
q_7	q_1	q_6	1

Moore makinesinin Durum Çizeneği:



Moore makinesinin denk durumları yoktur; indirgeme yapılamaz.

Y.1.15.

Denklik Bölümlemesi: $P = (ADE) (BG) (CF) (H) (J)$
 $S_0 \quad S_1 \quad S_2 \quad S_3 \quad S_4$

En küçük makinenin Durum Çizelgesi:

ŞD	SD, z	
	x = 0	x = 1
S_0	$S_2, 1$	$S_2, 0$
S_1	$S_3, 1$	$S_2, 0$
S_2	$S_0, 0$	$S_2, 1$
S_3	$S_0, 0$	$S_4, 0$
S_4	$S_1, 0$	$S_4, 1$

Y.1.16.

Denklik Bölümlemesi: $P = (A) (BE) (CDF) (G)$
 $S_0 \quad S_1 \quad S_2 \quad S_3$

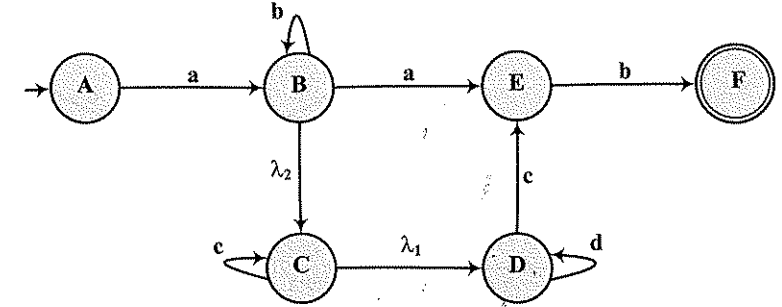
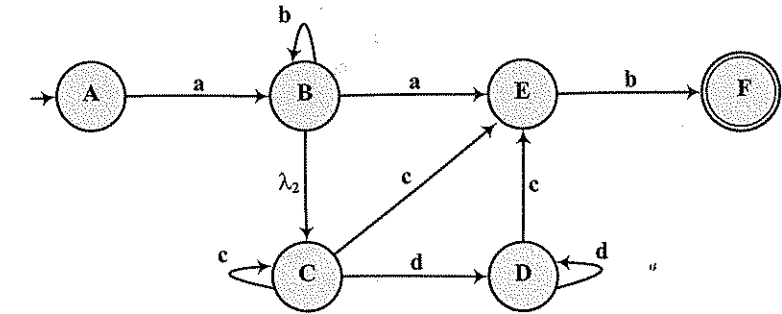
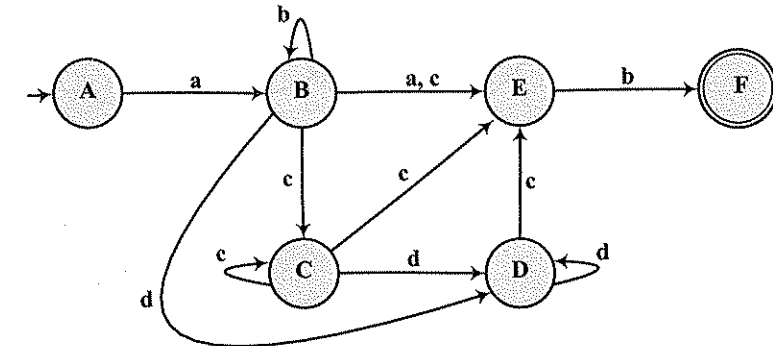
En küçük DFA'nın Geçiş Çizelgesi:

	0	1
$\rightarrow S_0$	S_0	S_1
S_1	S_2	S_1
S_2	S_2	S_3
$\odot S_3$	S_3	S_2

Bölüm 2

Y.2.1.

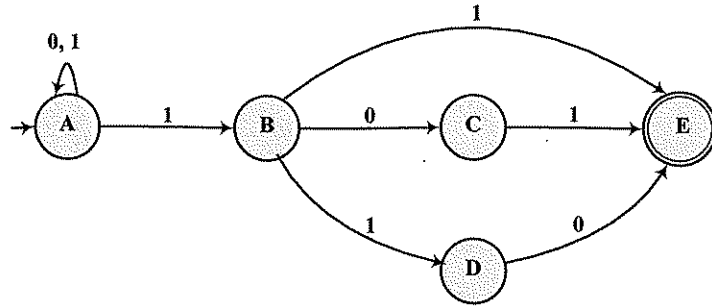
a)

b) λ_1 yok edildikten sonra: λ_2 yok edildikten sonra:

Y.2.2.

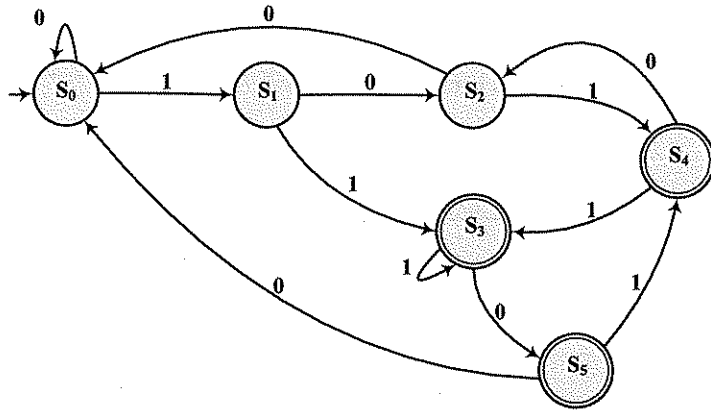
a) $L_{S.2.2} = (0+1)^* 1 (1+01+10)$

b) NFA'nın Geçiş Çizeneği:



c) DFA'nın Geçiş Çizelge ve Çizeneği:

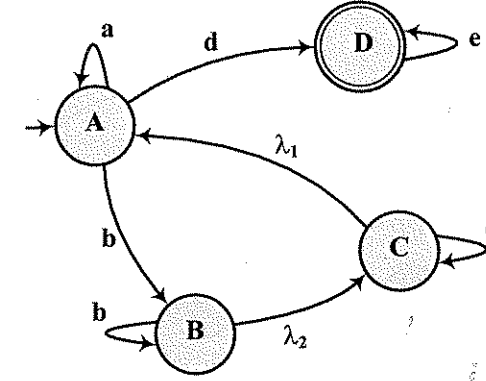
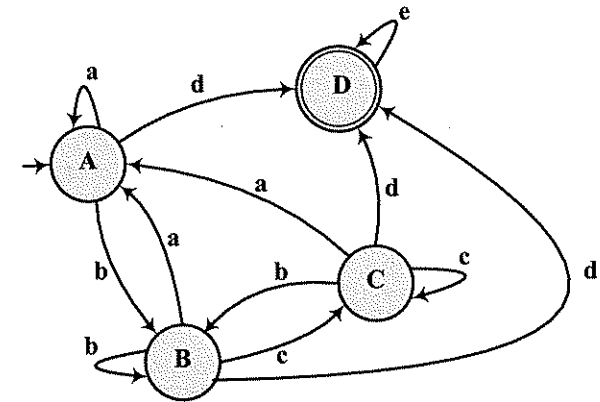
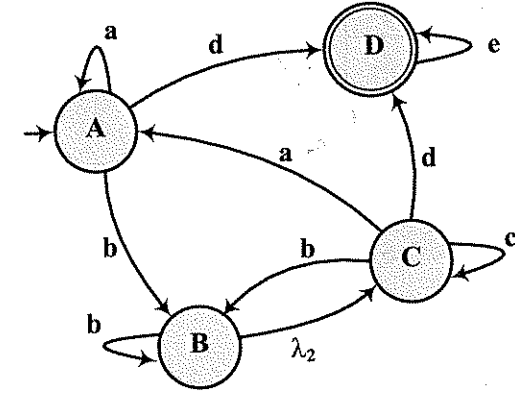
	0	1
$\rightarrow S_0$	S_0	S_1
S_1	S_2	S_3
S_2	S_0	S_4
S_3	S_5	S_3
S_4	S_2	S_3
S_5	S_0	S_4



d) Makinenin denk durumları yoktur; indirgeme yapılamaz.

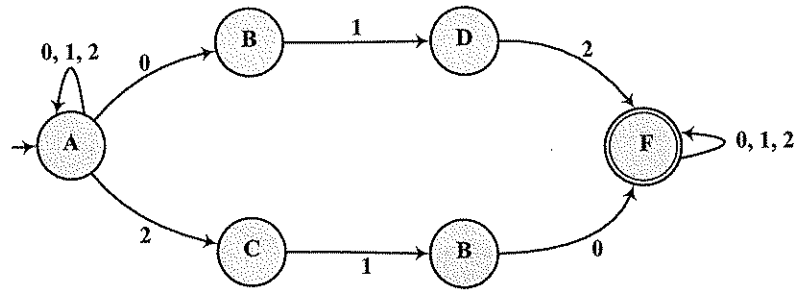
Y.2.3.

a) NFA'nın Geçiş Çizeneği:

b) λ -geçişsiz eşdeğer Geçiş Çizeneğinin bulunması:

Y.2.4.

a) $L_{S,2,4} = (0+1+2)^* (012+210)(0+1+2)^*$



b) DFA'nın Geçiş Çizelgesi:

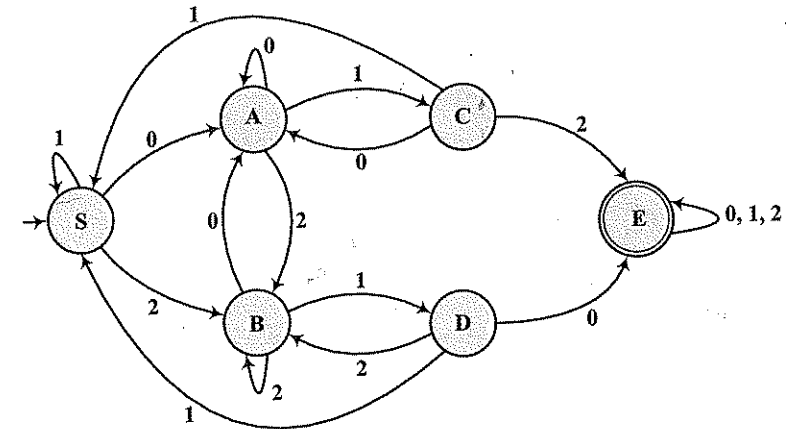
	0	1	2
$\rightarrow S_0$	S_1	S_0	S_2
S_1	S_1	S_3	S_2
S_2	S_1	S_4	S_2
S_3	S_1	S_0	S_5
S_4	S_6	S_0	S_2
S_5	S_6	S_7	S_5
S_6	S_6	S_8	S_5
S_7	S_6	S_9	S_5
S_8	S_6	S_9	S_5
S_9	S_6	S_9	S_5

c) Denklik Bölümlemesi : $P = (S_0) (S_1) (S_2) (S_3) (S_4) (S_5 S_6 S_7 S_8 S_9)$

S A B C D E

En küçük DFA'nın Geçiş Çizelge ve Çizeneği:

	0	1	2
$\rightarrow S$	A	S	B
A	A	C	B
B	A	D	B
C	A	S	E
D	E	S	B
E	E	E	E



Y.2.5.

a) $P_1 = (ca+bc+c)^*$

b) $P_2 = a^* (b+c) a^* (b+c) a^* (b+c) a^*$

c) $P_3 = (b+c)^* a (b+c)^* a (b+c)^* a (b+c)^*$

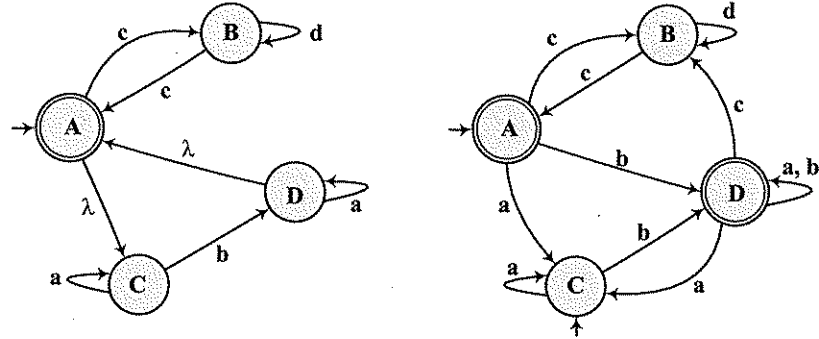
d) $P_4 = (b+c+ab+ac)^* (a+\lambda)$

e) $P_5 = (b+ab+aab)^* (a+aa+\lambda)$

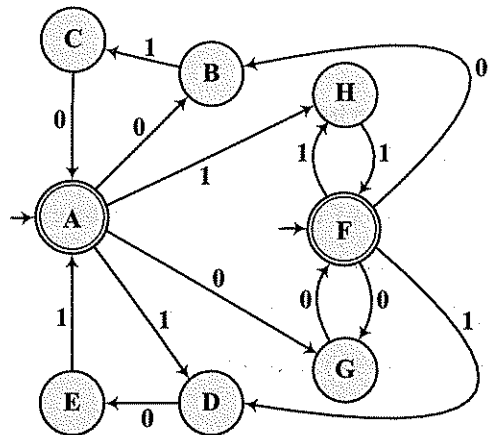
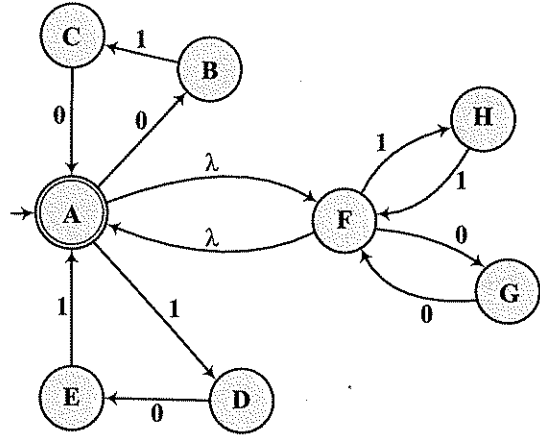
f) $P_6 = ((b+c)^* a (b+c)^* a (b+c)^*)^* (b+c)^* a (b+c)^* a (b+c)^*$

Y.2.6.

a)



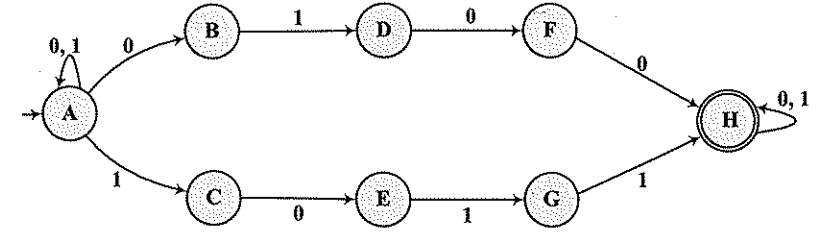
b)



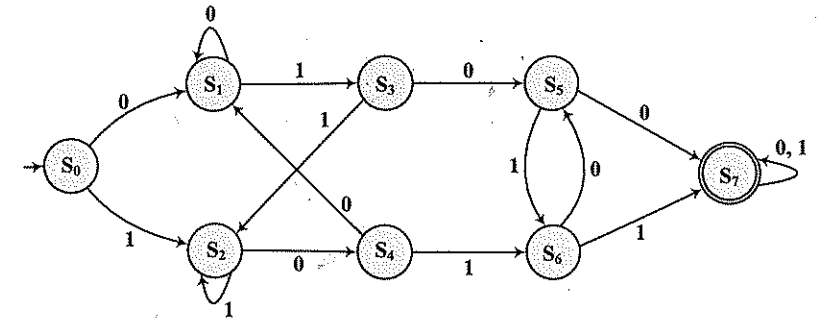
Y.2.7.

a) $L_{S.2.7} = (0+1)^*(0100+1011)(0+1)^*$

b)



c) En az durumlu denk DFA:



Y.2.8.

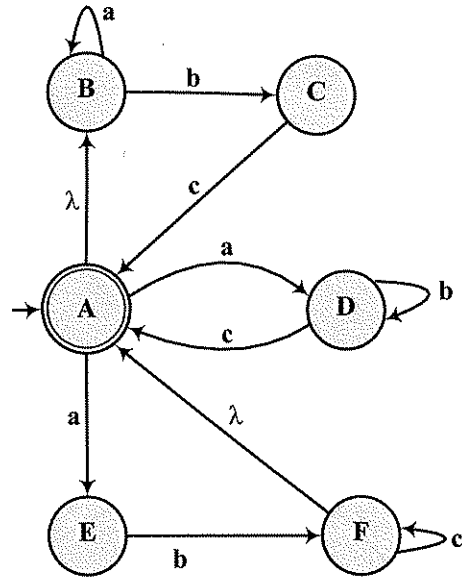
a) $P_1 = (b+c)^* a(b+c)^* a(b+c)^*$

b) $P_2 = (b^* ab^* ab^* ab^*)^* b^* ab^* ab^* ab^*$

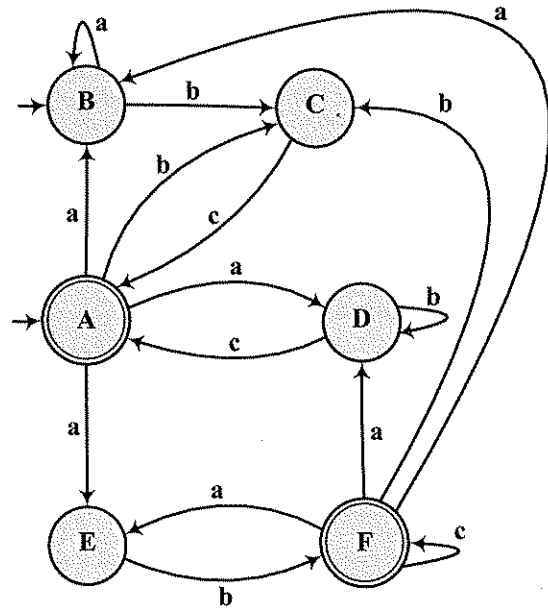
c) $P_3 = (b+ab)^*(a+\lambda)$

Y.2.9.

a)



b)



c)

DFA'nın Geçiş Çizelgesi:

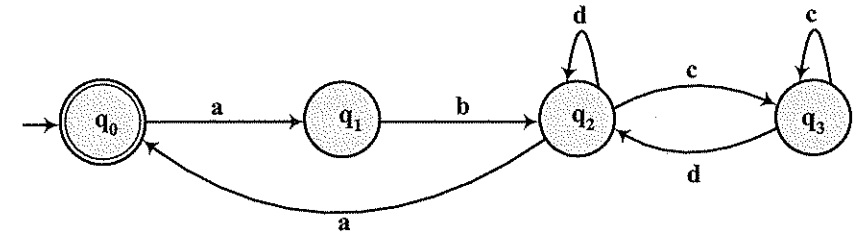
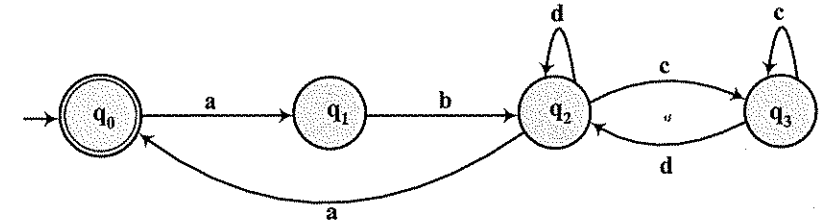
	a	b	c
→ q ₀	q ₁	q ₂	q ₃
q ₁	q ₄	q ₅	q ₀
q ₂	q ₃	q ₃	q ₀
q ₃	q ₃	q ₃	q ₃
q ₄	q ₄	q ₂	q ₃
q ₅	q ₁	q ₆	q ₇
q ₆	q ₃	q ₈	q ₀
q ₇	q ₁	q ₂	q ₉
q ₈	q ₃	q ₈	q ₀
q ₉	q ₁	q ₂	q ₉

Durumların karşılıkları:

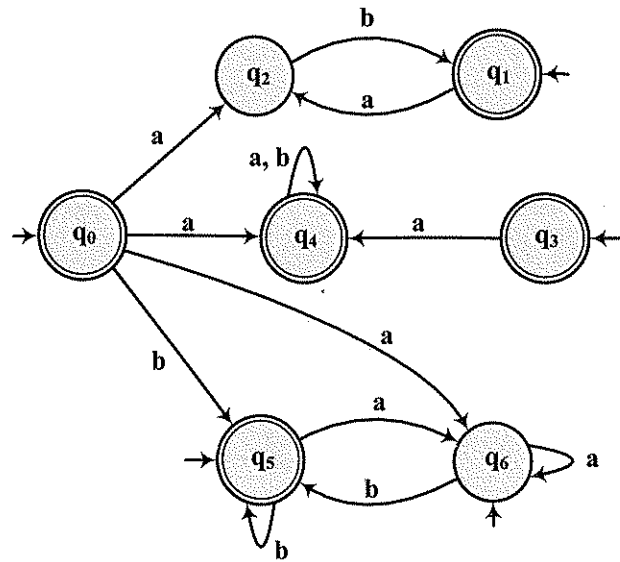
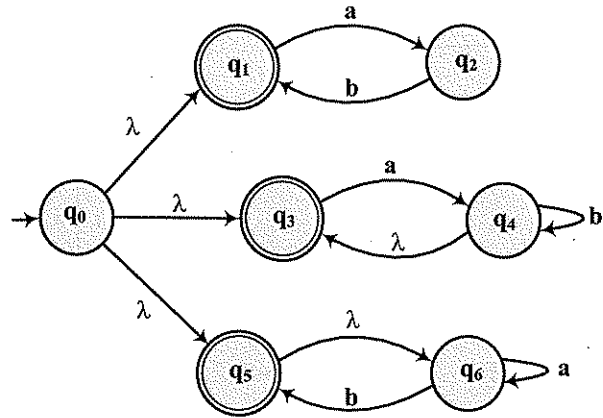
q₀ : Aq₁ : BDEq₂ : Cq₃ : Φq₄ : Bq₅ : CDFq₆ : CDq₇ : AFq₈ : Dq₉ : F

Y.2.10.

a)

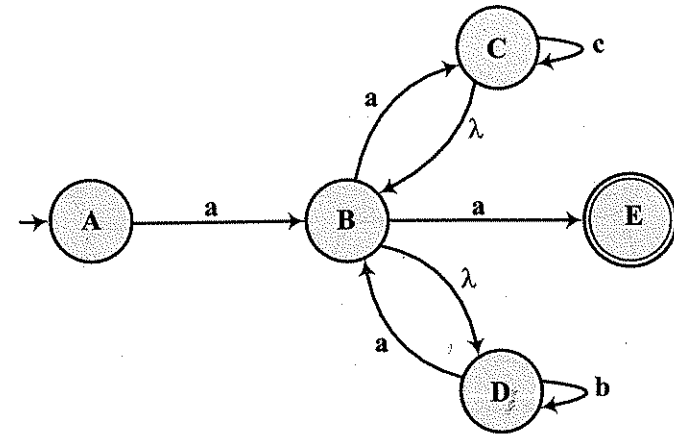


b)

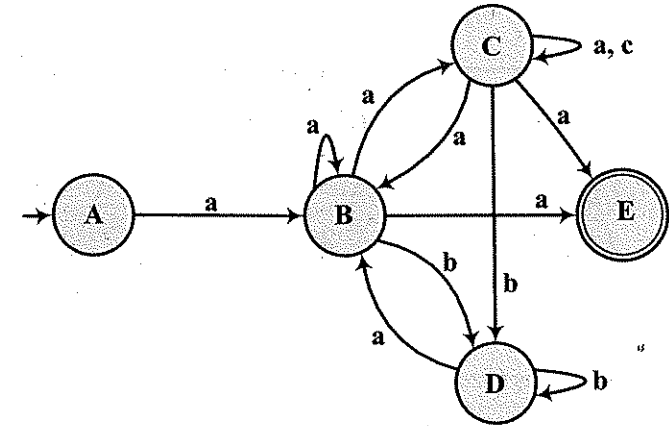


Y.2.11.

a)



b)



c) DFA'nın Geçiş Çizelgesi:

	a	b	c
$\rightarrow q_0$	q_1	q_2	q_2
q_1	q_3	q_4	q_2
q_2	q_2	q_2	q_2
$\odot q_3$	q_3	q_4	q_5
q_4	q_1	q_4	q_2
q_5	q_3	q_4	q_5

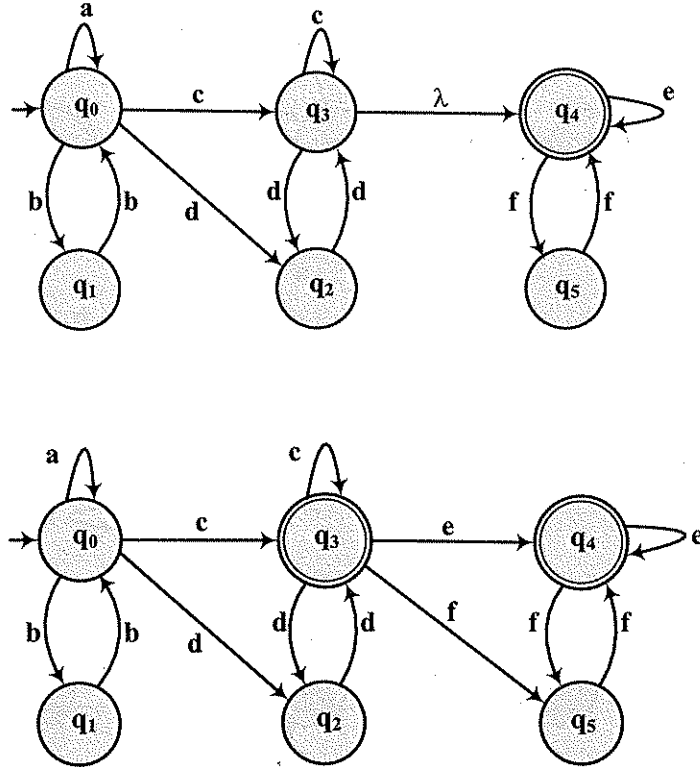
Durumların karşılıkları:

 q_0 : A q_1 : B q_2 : Φ q_3 : BCE q_4 : D q_5 : C

Y.2.12.

a) $L_{S.2.12} = (a+bb)^*(c+dd)(c+dd)^*(e+ff)^*$

b)

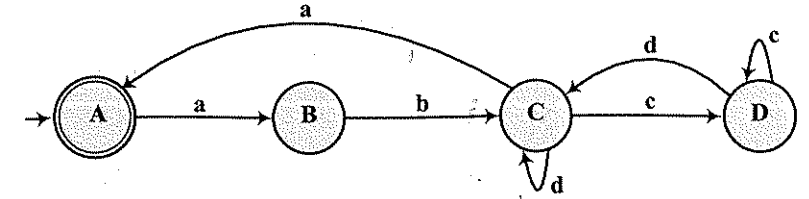
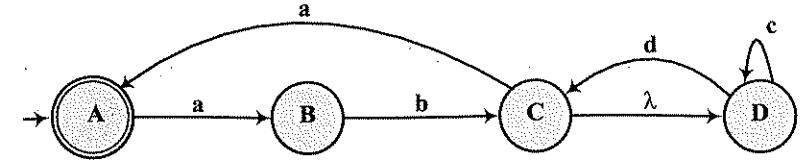


c) DFA'nın Geçiş Çizelgesi:

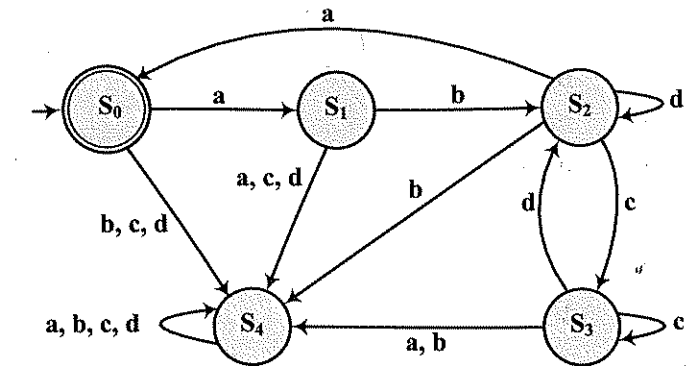
	a	b	c	d	e	f
→q ₀	q ₀	q ₁	q ₃	q ₂	q ₆	q ₆
q ₁	q ₆	q ₀	q ₆	q ₆	q ₆	q ₆
q ₂	q ₆	q ₆	q ₆	q ₃	q ₆	q ₆
q ₃	q ₆	q ₆	q ₃	q ₂	q ₄	q ₅
q ₄	q ₆	q ₆	q ₆	q ₆	q ₄	q ₅
q ₅	q ₆	q ₆	q ₆	q ₆	q ₆	q ₄
q ₆	q ₆	q ₆	q ₆	q ₆	q ₆	q ₆

Y.2.13.

a)



b)



Durumların karşılıkları:

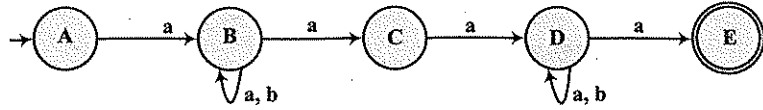
 S_0 : A S_1 : B S_2 : C S_3 : D S_4 : Φ

DFA en küçüktür; indirgeme yapılamaz.

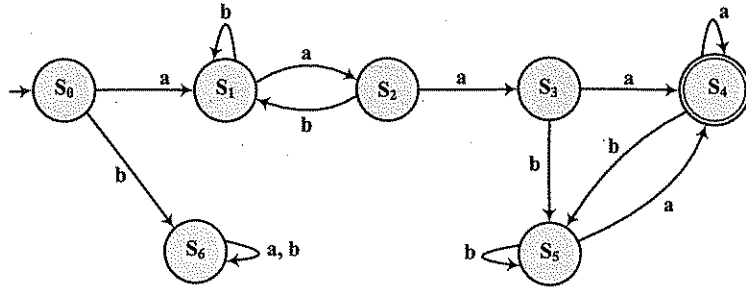
Y.2.14.

a) $L_{S.2.14} = a(a+b)^* aa(a+b)^* a$

b)



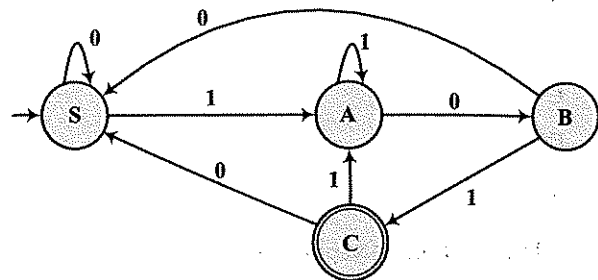
c)



Durumların karşılıkları:

 S_0 : A S_1 : B S_2 : BC S_3 : BCD S_4 : BCDE S_5 : BD S_6 : Φ Not: S_3 ve S_5 durumları denktir.

Y.2.15.



Denklemler Sistemi:

$$S = \lambda + S0 + B0 + C0$$

$$A = S1 + A1 + C1$$

$$B = A0$$

$$C = B1$$

$$L_{S.2.15} = (0+1(1+011)^*(00+010))^* 1(1+011)^* 01$$

Y.2.16. Denklemler Sistemi:

$$A = \lambda + A0 + B0 + C1$$

$$B = A1$$

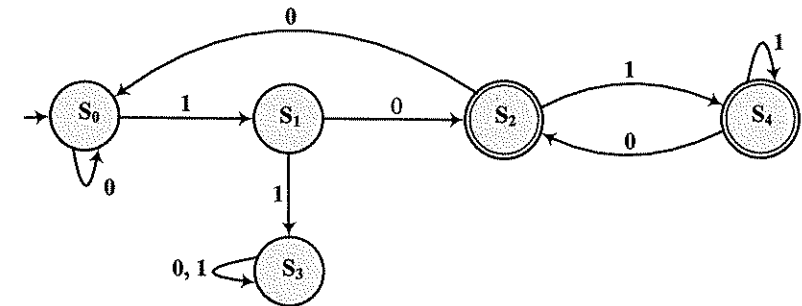
$$C = B0 + C1$$

$$L_{S.2.16} = (0+10+101^*1)^* 101^*$$

DFA'nın Geçiş Çizelgesi ve Çizenegi:

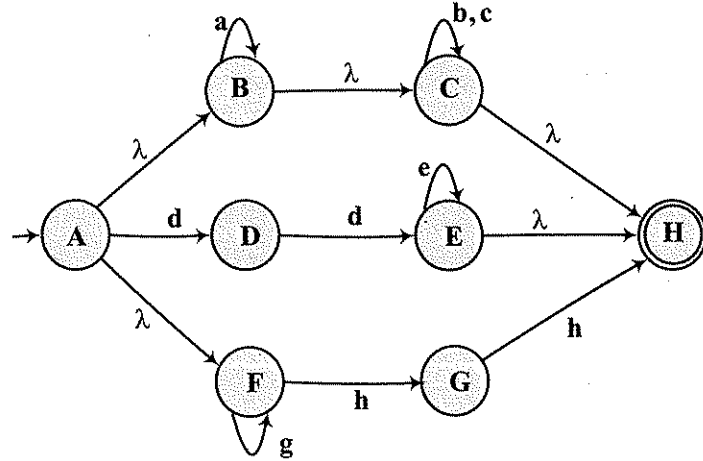
	0	1
$\rightarrow S_0$	S_0	S_1
S_1	S_2	S_3
$\odot S_2$	S_0	S_4
S_3	S_3	S_3
$\odot S_4$	S_2	S_4

Durumların karşılıkları:

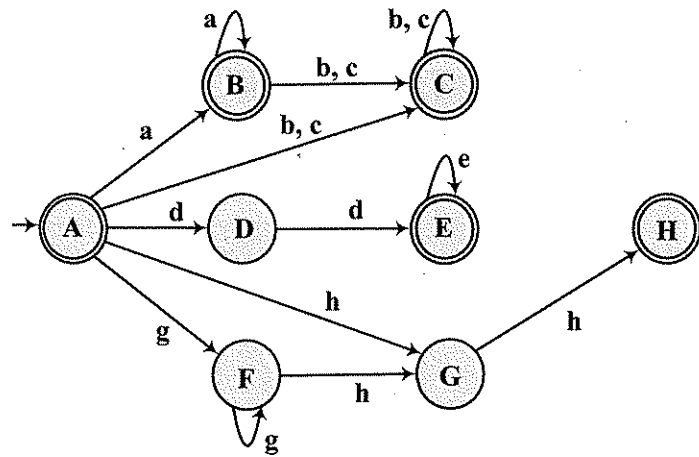
 S_0 : A S_1 : B S_2 : AC S_3 : Φ S_4 : ABC

Y.2.22. $L_{S.2.22} = a^*(b+c)^* + dde^* + g^*hh$

a)



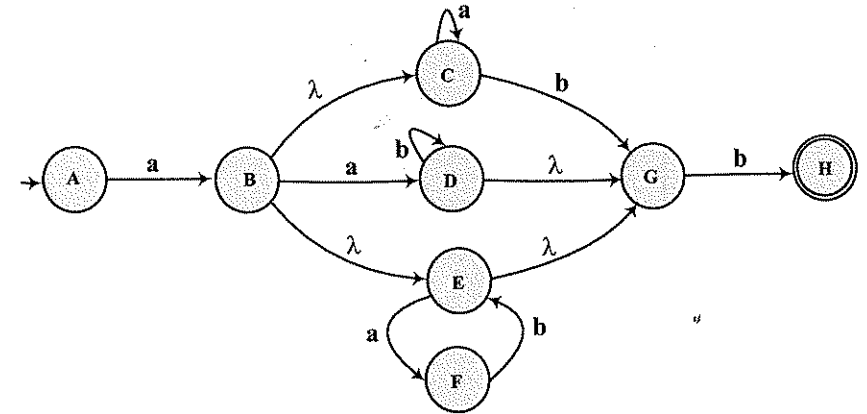
b) λ -geçişsiz denk Geçiş Çizeneği:



Geçiş Çizelgesi:

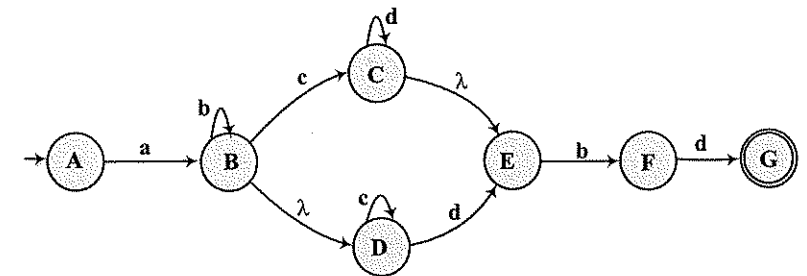
	a	b	c	d	e	g	h
→ A	B	C	C	D	-	F	G
B	B	C	C	-	-	-	-
C	-	C	C	-	-	-	-
D	-	-	-	E	-	-	-
E	-	-	-	-	E	-	-
F	-	-	-	-	-	F	G
G	-	-	-	-	-	-	H
H	-	-	-	-	-	-	-

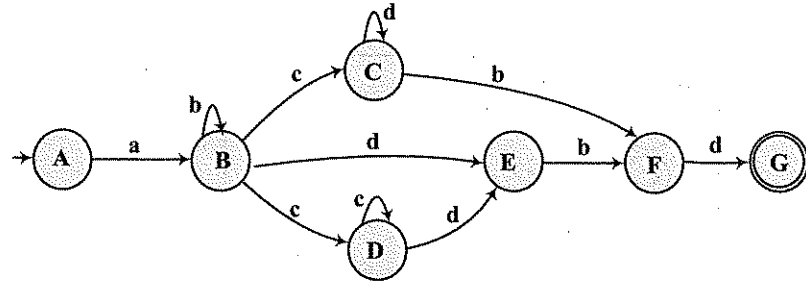
Y.2.24.



S.2.27.

a) Geçiş Çizeneği:



b) λ -Geçişsiz Denk Çizenek:

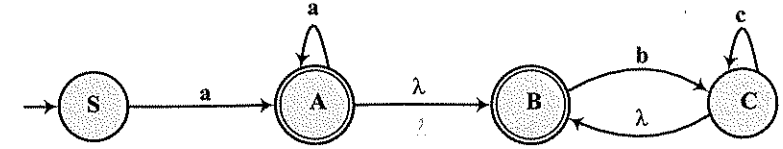
Geçiş Çizelgesi:

	a	b	c	d
→A	B	-	-	-
B	-	B	CD	E
C	-	F	-	C
D	-	-	D	E
E	-	F	-	-
F	-	-	-	G
⊙G	-	-	-	-

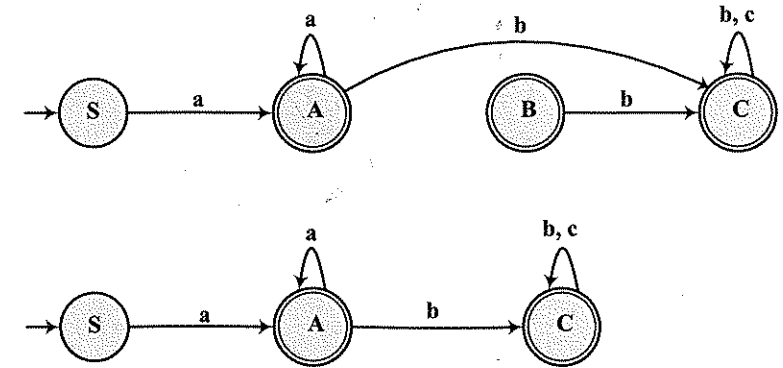
Bölüm 3

Y.3.1.

a)



b)

c) $G_{S.3.1} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, C\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow aA \mid a$$

$$A \Rightarrow aA \mid bC \mid a \mid b$$

$$C \Rightarrow bC \mid cC \mid b \mid c$$

Y.3.2.

a) $L_{S.3.2.1}$ düzgün bir dildir.

$$L_{S.3.2.1} = (b+c)^* abb((b+c)^* abb)^*(b+c)^*$$

b) $L_{S.3.2.2}$ düzgün bir dil değildir.

$$L_{S.3.2.2} = \{w w^R w w^R \mid w \in (a+b+c)^*\}$$

c) $L_{S.3.2.3}$ düzgün bir dil değildir.

$$L_{S.3.2.3} = \{a^n b^{n+k} c^k \mid n \geq 0, k \geq 0, n+k \geq 1\}$$

d) $L_{S.3.2.4}$ düzgün bir dildir.

$$L_{S.3.2.4} = ((a+c)^* b(a+c)^* b(a+c)^* b(a+c)^*)^* (a+c)^*$$

Y.3.3.

a) $L_{S.3.3.1}$ düzgün bir dildir.

$$G_{S.3.3.1} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{0, 1\}$$

$$P : S \Rightarrow 0S \mid 1A$$

$$A \Rightarrow 0A \mid 1B \mid 1$$

$$B \Rightarrow 0B \mid 1A \mid 0$$

b) $L_{S.3.3.2}$ bağlamdan-bağımsız bir dildir.

$$G_{S.3.3.2} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, D\}$$

$$V_T = \{a, b, c, d\}$$

$$P : S \Rightarrow aAcD$$

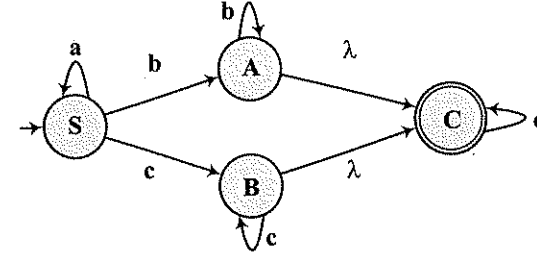
$$A \Rightarrow aAc \mid B$$

$$B \Rightarrow bB \mid b$$

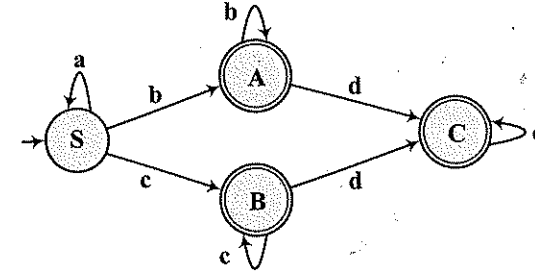
$$D \Rightarrow dD \mid d$$

Y.3.4.

a) $L_{S.3.4}$ dili sıfır, bir ya da birçok a ile başlayıp, en az bir b ya da c ile devam eden ve sonunda sıfır, bir ya da birçok d bulunana dizgiler kümesidir.



b)



c) $G_{S.3.4} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{a, b, c, d\}$$

$$P : S \Rightarrow aS \mid bA \mid cB \mid b \mid c$$

$$A \Rightarrow bA \mid dC \mid b \mid d$$

$$B \Rightarrow cB \mid dC \mid c \mid d$$

$$C \Rightarrow dC \mid d$$

Y.3.5.

a) $L_{S.3.5.1}$ bağlamdan-bağımsız bir dildir.

$$G_{S.3.5.1} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b, c, d\}$$

$$P : S \Rightarrow aAd$$

$$A \Rightarrow aAd \mid Ad \mid bBcc$$

$$B \Rightarrow bBc \mid Bc \mid \lambda$$

b) $L_{S.3.5.2}$ düzgün bir dildir.

$$G_{S.3.5.2} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P : S \Rightarrow ababA$$

$$A \Rightarrow abA \mid babaB$$

$$B \Rightarrow baB \mid \lambda$$

c) $L_{S.3.5.3}$ bağlamdan-bağımsız bir dildir.

$$G_{S.3.5.3} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S\}$$

$$V_T = \{0, 1\}$$

$$P : S \Rightarrow 0S0 \mid 1S1 \mid \lambda$$

Y.3.6.

a) $L_{S.3.6} = \{w0(w^R)' + w1(w^R)' \mid w \in (0+1)^*\}$

Not : $(w^R)' = w$ nin tersinin 1'e tümleridir.

b) $G_{S.3.6} = \langle V_N, V_T, P, S \rangle$

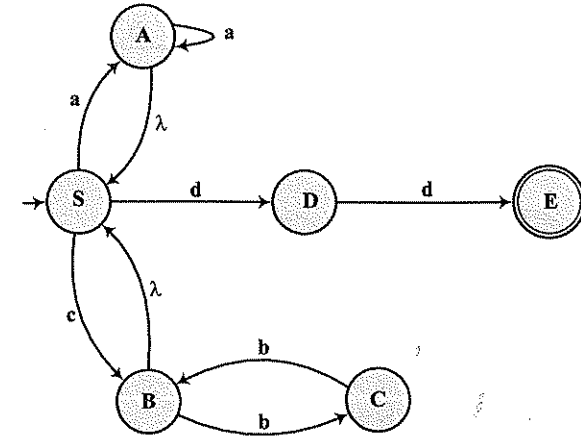
$$V_N = \{S\}$$

$$V_T = \{0, 1\}$$

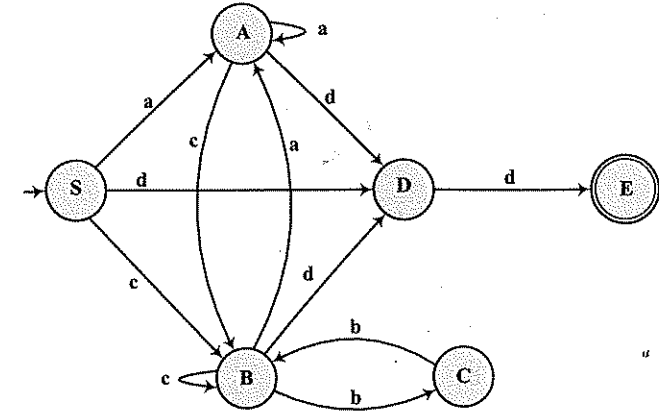
$$P : S \Rightarrow 0S1 \mid 1S0 \mid 0 \mid 1$$

Y.3.7.

a)



b)



c) $G_{S.3.7} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B, C, D\}$$

$$V_T = \{a, b, c, d\}$$

$$P : S \Rightarrow aA \mid cB \mid dD$$

$$A \Rightarrow aA \mid cB \mid dD$$

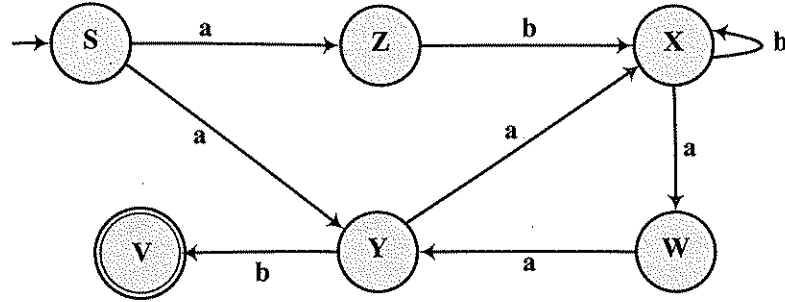
$$B \Rightarrow aA \mid cB \mid dD \mid bC$$

$$C \Rightarrow bB$$

$$D \Rightarrow d$$

Y.3.8.

a)



$$b) L(G_{S.3.8}) = (a+abb^*aa)(ab^*aa)^*b$$

Y.3.9.

- a) $L_{S.3.9.1} = \{a^n b^n c^m \mid n, m \geq 1\}$
 b) $L_{S.3.9.2} = \{a^n c^m b^k \mid n, m \geq 1, n \geq k \geq 2n\}$
 c) $L_{S.3.9.3} = \{a^n b^k c^k d^n \mid n, k \geq 1\}$

Y.3.10.

a) $L_{S.3.10.1}$ düzgün bir dildir.

$$G_{S.3.10.1} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P : S \Rightarrow aA$$

$$A \Rightarrow aA \mid bB$$

$$B \Rightarrow bB \mid \lambda$$

b) $L_{S.3.10.2}$ bağlamdan-bağımsız bir dildir.

$$G_{S.3.10.2} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A\}$$

$$V_T = \{a, b\}$$

$$P : S \Rightarrow aAb \mid aAbb$$

$$A \Rightarrow aAb \mid aAbb \mid ab \mid abb$$

c) $L_{S.3.10.3}$ bağlamdan-bağımsız bir dildir.

$$G_{S.3.10.3} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{0, 1\}$$

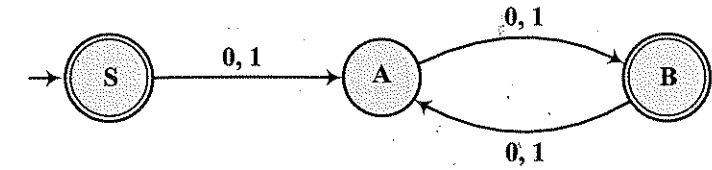
$$P : S \Rightarrow AB$$

$$A \Rightarrow 0A0 \mid 1A1 \mid \lambda$$

$$B \Rightarrow 0B0 \mid 1B1 \mid \lambda$$

Y.3.11.

a)

b) $G_{S.3.11} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S\}$$

$$V_T = \{0, 1\}$$

$$P : S \Rightarrow 00S \mid 01S \mid 10S \mid 11S \mid \lambda$$

Y.3.12.

a) $G_{S.3.12.1} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B, C\}$$

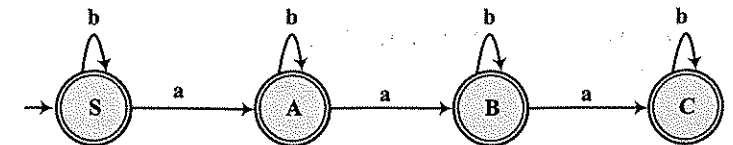
$$V_T = \{a, b\}$$

$$P : S \Rightarrow bS \mid aA \mid \lambda$$

$$A \Rightarrow bA \mid aB \mid \lambda$$

$$B \Rightarrow bB \mid aC \mid \lambda$$

$$C \Rightarrow bC \mid \lambda$$

 $G_{S.3.12.1}$ düzgün bir dilbilgisidir.

$$b) G_{S.3.12.2} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow aaaA$$

$$A \Rightarrow aAb \mid \lambda$$

$G_{S.3.12.2}$ bağlamdan-bağımsız bir dilbilgisidir.

$$c) G_{S.3.12.3} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow aAbb \mid aAbbb$$

$$A \Rightarrow aAbb \mid aAbbb \mid \lambda$$

$G_{S.3.12.3}$ bağlamdan-bağımsız bir dilbilgisidir.

$$Y.3.13. \quad w_1 = aa$$

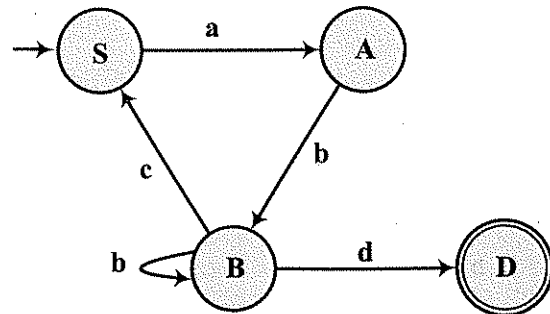
$$w_2 = aaaaa$$

$$w_3 = aaaaaaaaa$$

$$L(G_{S.3.13}) = \{w \mid w \in a^+ \text{ ve } |w| = 2^n, n \geq 1\}$$

Y.3.14.

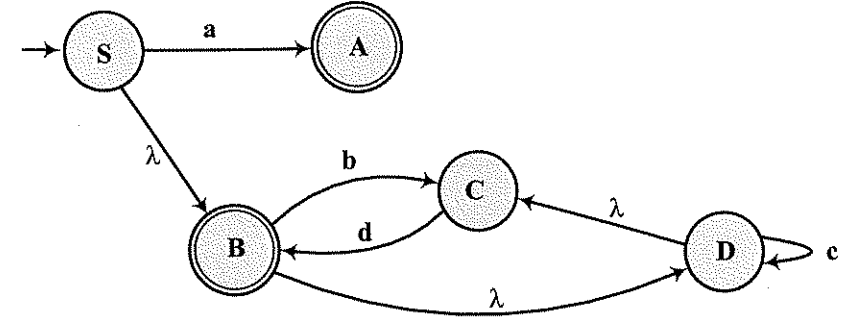
a)



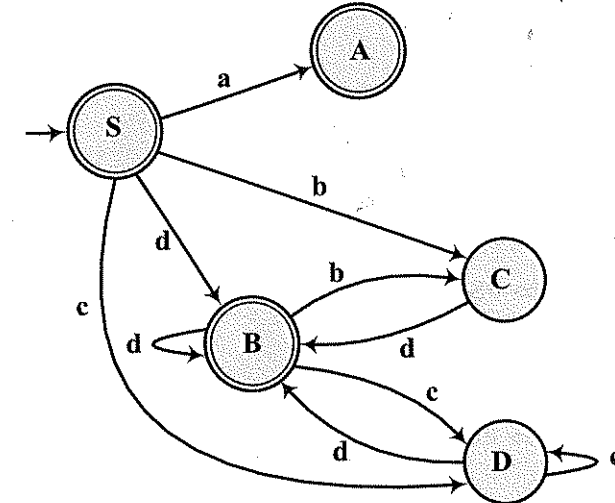
$$b) L(G_{S.3.14}) = (abb^*c)^*abb^*d$$

Y.3.15.

a)



b)



$$c) G_{S.3.15} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C, D\}$$

$$V_T = \{a, b, c, d\}$$

$$P: S \Rightarrow aA \mid dB \mid bC \mid cD \mid a \mid d \mid \lambda$$

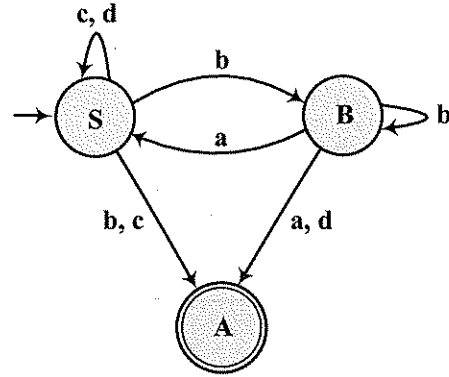
$$B \Rightarrow dB \mid bC \mid cD \mid d$$

$$C \Rightarrow dB \mid d$$

$$D \Rightarrow dB \mid cD \mid d$$

Y.3.16.

a)



$$b) L(G_{S.3.16}) = (c+d+bb^*a)^*(b+c+bb^*(a+d))$$

Y.3.17.

$$a) L_{S.3.17} = (0+1)^*0(0+1)0(0+1)^*$$

$$b) G_{S.3.17} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0S \mid 1S \mid 0A$$

$$A \Rightarrow 0B \mid 1B$$

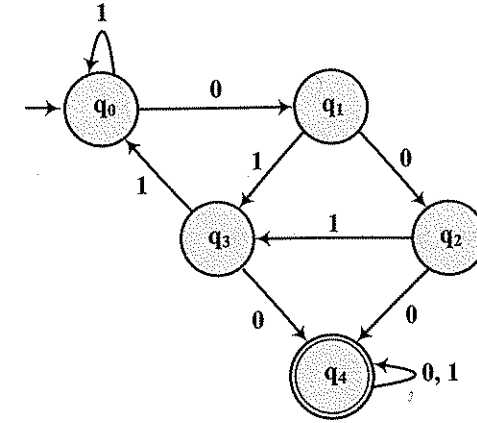
$$B \Rightarrow 0C \mid 0$$

$$C \Rightarrow 0C \mid 1C \mid 0 \mid 1$$

c)

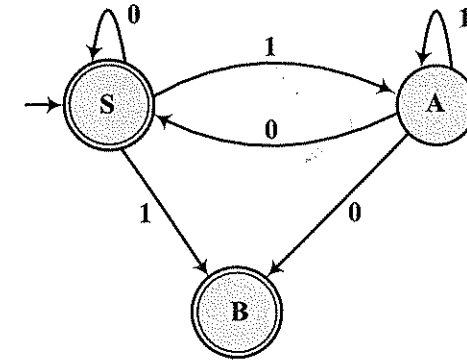
	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_3
q_2	q_4	q_3
q_3	q_4	q_0
q_4	q_4	q_4

Durumların karşılıkları:

 q_0 : S q_1 : SA q_2 : SAB q_3 : SB q_4 : SC, SAC, SBC, SABC

Y.3.18.

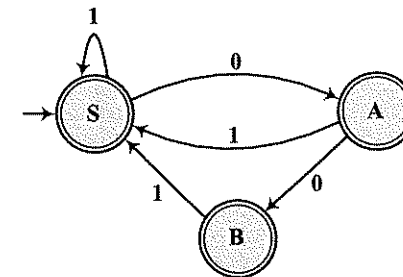
a)



$$b) L(G_{S.3.18}) = (0+11^*0)^*(\lambda+1+11^*0)$$

Y.3.19.

a)



Tanımlanan kümeyi türeten sağ-doğrusal bir dilbilgisi:

$$G_{S.3.19.1} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

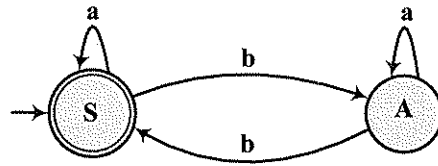
$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 1S \mid 0A \mid 0 \mid 1 \mid \lambda$$

$$A \Rightarrow 1S \mid 0B \mid 0 \mid 1$$

$$B \Rightarrow 1S \mid 1$$

b)



Tanımlanan kümeyi türeten sağ-doğrusal bir dilbilgisi:

$$G_{S.3.19.2} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A\}$$

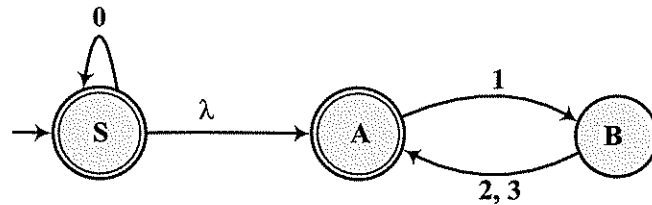
$$V_T = \{a, b\}$$

$$P: S \Rightarrow aS \mid bA \mid a \mid \lambda$$

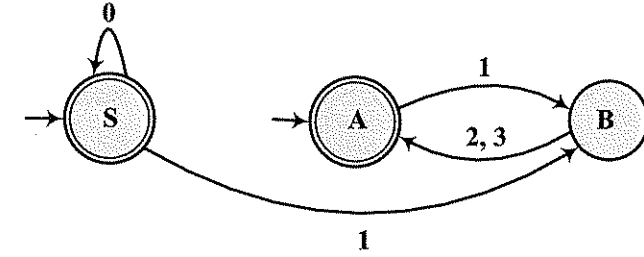
$$A \Rightarrow bS \mid aA \mid b$$

Y.3.20.

a)



b)



$$c) G_{S.3.20} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{0, 1, 2, 3\}$$

$$P: S \Rightarrow 0S \mid 1B \mid 0 \mid \lambda$$

$$A \Rightarrow 1B$$

$$B \Rightarrow 2A \mid 3A \mid 2 \mid 3$$

$$d) G_{S.3.20.2} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A\}$$

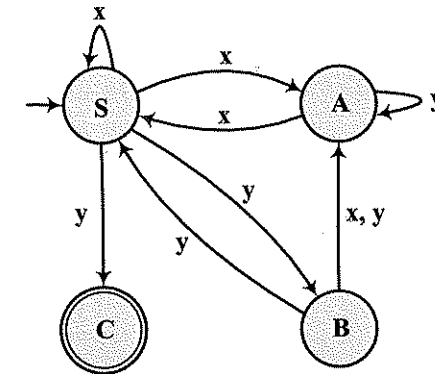
$$V_T = \{0, 1, 2, 3\}$$

$$P: S \Rightarrow S12 \mid S13 \mid A \mid \lambda$$

$$A \Rightarrow A0 \mid 0$$

Y.3.21.

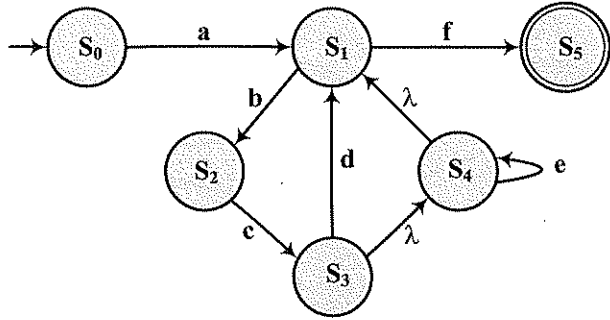
a)



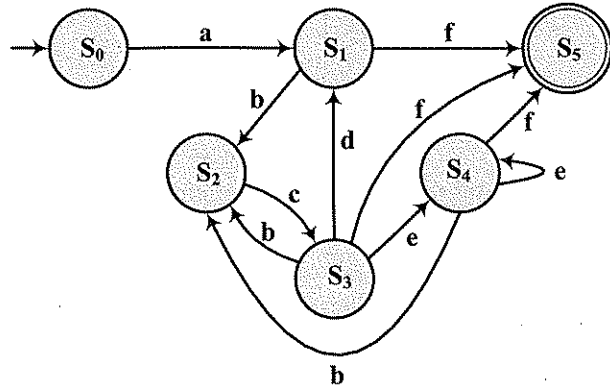
$$b) L(G_{S.3.21}) = (x+yy+(x+yx+yy)y^*x)^*y$$

Y.3.22.

a)



b)

c) $G_{S.3.22} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S_0, S_1, S_2, S_3, S_4\}$$

$$V_T = \{a, b, c, d, e, f\}$$

$$P : S_0 \Rightarrow aS_1$$

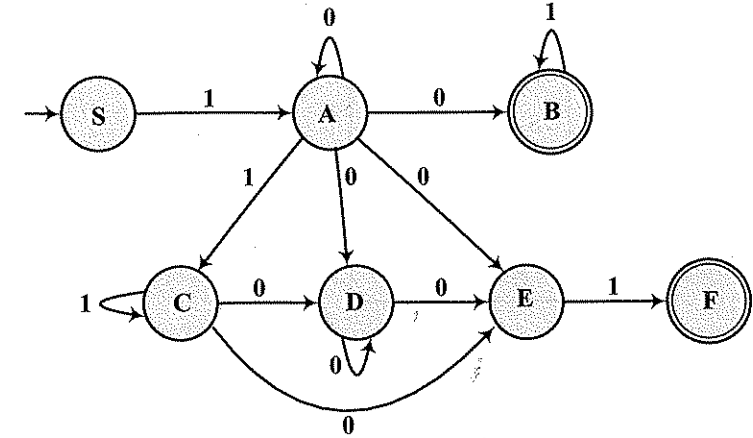
$$S_1 \Rightarrow bS_2 \mid f$$

$$S_2 \Rightarrow cS_3$$

$$S_3 \Rightarrow dS_1 \mid bS_2 \mid eS_4 \mid f$$

$$S_4 \Rightarrow bS_2 \mid eS_4 \mid f$$

Y.3.23.

a) λ -geçişsiz geçiş çizeneği:b) $G_{S.3.23} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B, C, D, E\}$$

$$V_T = \{0, 1\}$$

$$P : S \Rightarrow 1A$$

$$A \Rightarrow 0A \mid 0B \mid 1C \mid 0D \mid 0E \mid 0$$

$$B \Rightarrow 1B \mid 1$$

$$C \Rightarrow 1C \mid 0D \mid 0E$$

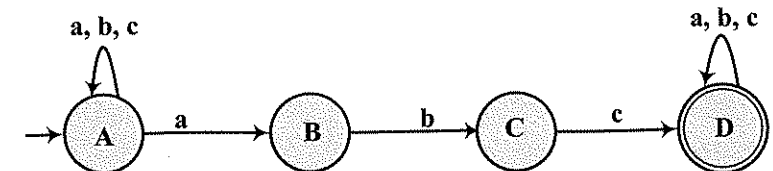
$$D \Rightarrow 0D \mid 0E$$

$$E \Rightarrow 1$$

$$Y.3.24. \quad L(G_{S.3.24}) = (0+10^*10)(00^*10)^*0+(1+00)(0+110)^*1$$

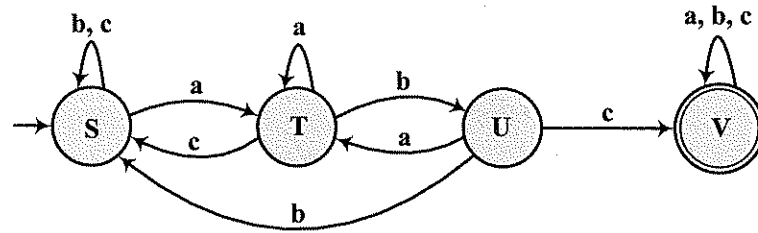
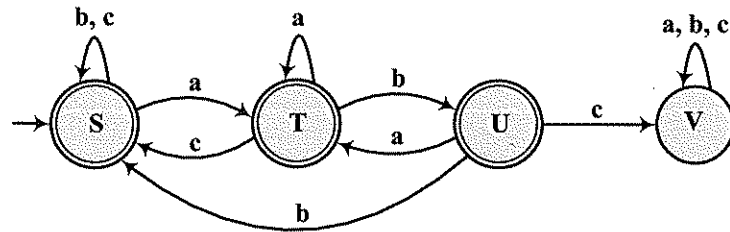
Y.3.25.

a)



b)

	a	b	c
$\rightarrow S_0$	S_1	S_0	S_0
S_1	S_1	S_2	S_0
S_2	S_1	S_0	S_3
S_3	S_4	S_3	S_3
S_4	S_4	S_5	S_3
S_5	S_4	S_3	S_3

c) Denklik bölümlemesi: $P = (S_0) (S_1) (S_2) (S_3 S_4 S_5)$
 $S \quad T \quad U \quad V$ d) $L'_{S.3.25} = (a+b+c)^* - L_{S.3.25}$ $L'_{S.3.25}$ dilini tanıyan DFA'nın geçiş çizeneği:e) $G'_{S.3.25} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, T, U, V\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow bS \mid cS \mid aT \mid a \mid b \mid c \mid \lambda$$

$$T \Rightarrow aT \mid bU \mid cS \mid a \mid b \mid c$$

$$U \Rightarrow bS \mid aT \mid cV \mid a \mid b$$

$$V \Rightarrow aV \mid bV \mid cV$$

V yararsız değişkendir. İçinde V bulunan kurallar da yararsız kurallardır.

Y.3.26.

a) $G_{S.3.26} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow aA \mid aB$$

$$A \Rightarrow aA \mid bC \mid b$$

$$B \Rightarrow aC \mid a$$

$$C \Rightarrow bB$$

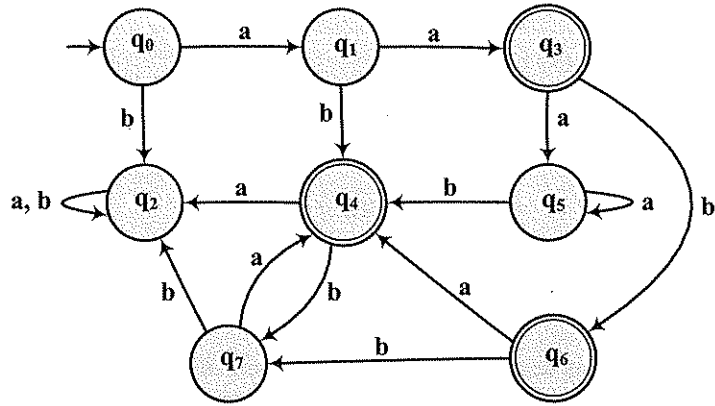
b) $L(G_{S.3.26}) = (aa^*b+aa)(ba)^*$

c)

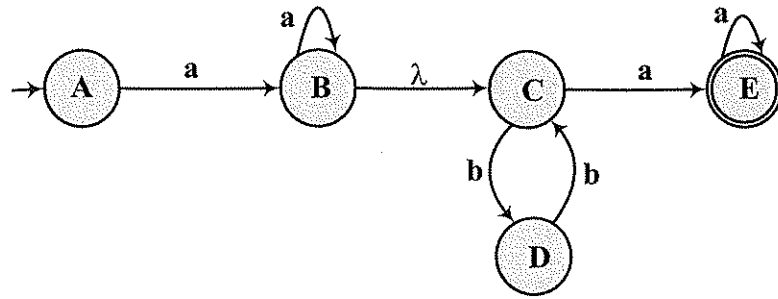
	a	b
$\rightarrow q_0$	q_1	q_2
q_1	q_3	q_4
q_2	q_2	q_2
q_3	q_5	q_6
q_4	q_2	q_7
q_5	q_5	q_4
q_6	q_4	q_7
q_7	q_4	q_2

Durumların karşılıkları:

 $q_0 : S$ $q_1 : AB$ $q_2 : \Phi$ $q_3 : AC$ $q_4 : C$ $q_5 : A$ $q_6 : BC$ $q_7 : B$



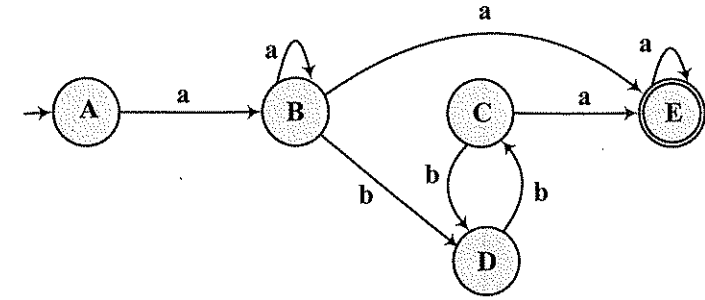
Y.3.27.



a) $L_{S.3.27} = aa^*(bb)^*aa^*$

b) Bir seçenek : $S \Rightarrow Sa \mid Aa$
 $A \Rightarrow Abb \mid Ba \mid a$
 $B \Rightarrow Ba \mid a$

Bir diğer seçenek : $S \Rightarrow ABA \mid AA$
 $A \Rightarrow Aa \mid a$
 $B \Rightarrow Bbb \mid bb$

c) λ -geçişsiz çizenecek:

Ardıl Çizelgesi:

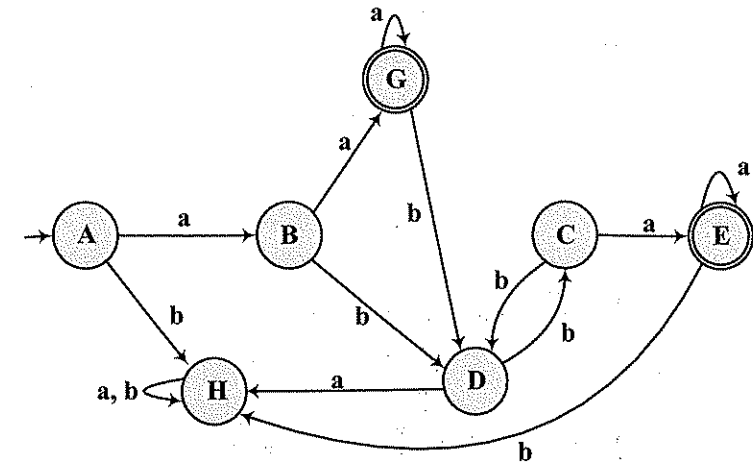
	a	b
A	B	-
B	BE	D
C	E	E
D	-	C
E	E	-

Geçiş çizeneğinin deterministik olmasını sağlamak için 2 durum eklemek gerekir:

BE için : G

C için : H

Deterministik Çizenecek:



DFA'nın Geçiş Çizelgesi:

	a	b
→ A	B	H
B	G	D
C	E	D
D	H	C
⊙ E	E	H
⊙ G	G	D
H	H	H

Y.3.33. $G_{S.3.33} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow A1B1B1A$$

$$A \Rightarrow A0 \mid \lambda$$

$$B \Rightarrow B0 \mid 0$$

Y.3.34.

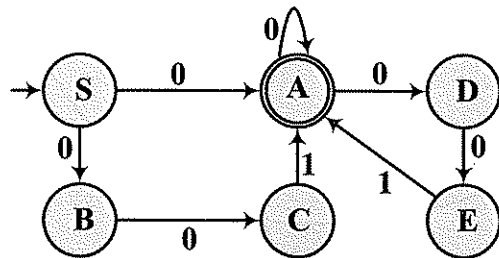
a) $G_{S.3.34} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0A \mid 001A$$

$$A \Rightarrow 0A \mid 001A \mid \lambda$$

b) $L_{S.3.34}$ dilini tanıyan sonlu özdevinirin (NFA) Geçiş Çizeneği:

Y.3.35.

a) $G_{S.3.35} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0S \mid 010S \mid 0 \mid 010$$

b) $G'_{S.3.35} = \langle V_N, V_T, P, S \rangle$

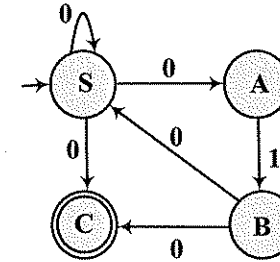
$$V_N = \{S, A, B\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0S \mid 0A \mid 0$$

$$A \Rightarrow 1B$$

$$B \Rightarrow 0S \mid 0$$

c) $L(G_{S.3.35})$ 'i tanıyan FA'nın Geçiş Çizeneği:

Bölüm 4

Y.4.1.

a) $G_{S.4.1.1} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B, C, D\}$$

$$V_T = \{0, 1, 2\}$$

$$P: S \Rightarrow AB \mid CD$$

$$A \Rightarrow 0A1 \mid 01$$

$$B \Rightarrow 2B \mid 2$$

$$C \Rightarrow 0C \mid 0$$

$$D \Rightarrow 1D2 \mid 12$$

b) $G_{S.4.1.2} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow bS \mid aB \mid bA \mid b$$

$$A \Rightarrow aS \mid bAA \mid a \mid \lambda$$

$$B \Rightarrow bS \mid aBB \mid b$$

c) $G_{S.4.1.3} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S\}$$

$$V_T = \{a, b, c, d\}$$

$$P: S \Rightarrow bS \mid abS \mid cddS \mid dS \mid b \mid ab \mid cdd \mid d$$

Y.4.2.

a) $L_{S.4.2} = \{(aa)^n (cc)^m b^k c b^k \mid n \geq 1, m \geq 0, k \geq 1\}$

b) $G_{S.4.2} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B, C, D, E, X_1, X_2, X_3, X_4\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow AX_1 \mid AX_2$$

$$A \Rightarrow a$$

$$B \Rightarrow b$$

$$C \Rightarrow c$$

$$D \Rightarrow BX_3 \mid CX_4$$

$$E \Rightarrow BX_3 \mid c$$

$$X_1 \Rightarrow AS$$

$$X_2 \Rightarrow AD$$

$$X_3 \Rightarrow EB$$

$$X_4 \Rightarrow CD$$

Y.4.3.

a) $G_{S.4.3.1}$ bağlama-bağımlı (tür-1) bir dilbilgisidir.

$$L_{S.4.3.1} = \{a^i b^i c^i \mid i \geq 1\}$$

 $L_{S.4.3.1}$ bağlamdan-bağımsız bir dilbilgisi ile türetilemez.b) $G_{S.4.3.2}$ bağlama-bağımlı (tür-1) bir dilbilgisidir.

$$L_{S.4.3.2} = \{a^{n+1} b^{2n} \mid n \geq 0\}$$

 $L_{S.4.3.2}$ aşağıdaki bağlamdan-bağımsız dilbilgisi ile türetilir.

$$G = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow aA \mid a$$

$$A \Rightarrow aAbb \mid abb \mid \lambda$$

Y.4.4. $G_{S.4.4} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B, C, D, E, T, U, V, W\}$$

$$V_T = \{a, b, d, e\}$$

$$P: S \Rightarrow AT \mid BU \mid DE$$

$$A \Rightarrow a$$

$$B \Rightarrow b$$

$$C \Rightarrow AW \mid b$$

$$D \Rightarrow d$$

$$E \Rightarrow e$$

$$T \Rightarrow BC$$

$$U \Rightarrow AV$$

$$V \Rightarrow BS$$

$$W \Rightarrow CA$$

Y.4.5.

$$a) G_{S.4.5.1} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A\}$$

$$V_T = \{a, b, c, d\}$$

$$P: S \Rightarrow aaSd \mid aaAd$$

$$A \Rightarrow bAcc \mid bcc$$

$$b) G_{S.4.5.2} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b, c, d\}$$

$$P: S \Rightarrow aSdd \mid A$$

$$A \Rightarrow aAc \mid B$$

$$B \Rightarrow bBc \mid \lambda$$

Y.4.6.

$$a) L_{S.4.6.1} = \{a^n b a^m b a^k \mid n \geq 1, m \geq 0, k \geq 1\}$$

$$b) L_{S.4.6.2} = \{a^{n+m} b c^{m+k} d e^{k+n} \mid n, m, k \geq 0\}$$

Y.4.7.

$$a) G_{S.4.7.1} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow aaAc$$

$$A \Rightarrow aAc \mid aA \mid aBb$$

$$B \Rightarrow aBb \mid aB \mid \lambda$$

$$b) G_{S.4.7.2} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow aAcc$$

$$A \Rightarrow aAcc \mid bBc$$

$$B \Rightarrow bBc \mid \lambda$$

Y.4.8.

$$a) L_{S.4.8.1} = \{a^n b^{2k+1} d^{2k+1} a^{2n} \mid n \geq 0, k \geq 0\}$$

$$b) L_{S.4.8.2} = \{(ab)^n (cd)^k (ba)^k (dc)^n \mid n \geq 0, k \geq 0\}$$

Y.4.9. Yararsız değişkenler: B, C, E, G

Yararsız uç simgeler: c, e

Eşdeğer dilbilgisi:

$$G_{S.4.9} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, D, F\}$$

$$V_T = \{a, b, d\}$$

$$P: S \Rightarrow dA$$

$$A \Rightarrow dA \mid dD$$

$$D \Rightarrow aD \mid aF \mid a$$

$$F \Rightarrow dF \mid b$$

$$L(G_{S.4.9}) = dd^* da^* a + dd^* da^* ad^* b$$

Y.4.10.

$$a) G_{S.4.10.1} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b, c, d\}$$

$$P: S \Rightarrow aAd$$

$$A \Rightarrow aAd \mid B$$

$$B \Rightarrow bBc \mid bc$$

b) $G_{S.4.10.2} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow aSc \mid A$$

$$A \Rightarrow aAb \mid ab$$

c) $G_{S.4.10.3} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A\}$$

$$V_T = \{b, c\}$$

$$P: S \Rightarrow AbSc \mid Abc$$

$$A \Rightarrow b \mid \lambda$$

d) $G_{S.4.10.4} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{a, b, c, d\}$$

$$P: S \Rightarrow ABC$$

$$A \Rightarrow aA \mid \lambda$$

$$B \Rightarrow bBc \mid bc$$

$$C \Rightarrow dC \mid \lambda$$

Y.4.11. $G_{S.4.11} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A_1, A_2, A_3, A_4, B_1, B_2, B_3\}$$

$$V_T = \{\sim, \supset, [,], p, q\}$$

$$P: S \Rightarrow A_1S \mid B_1A_2 \mid p \mid q$$

$$A_1 \Rightarrow \sim$$

$$A_2 \Rightarrow]$$

$$B_1 \Rightarrow B_2S$$

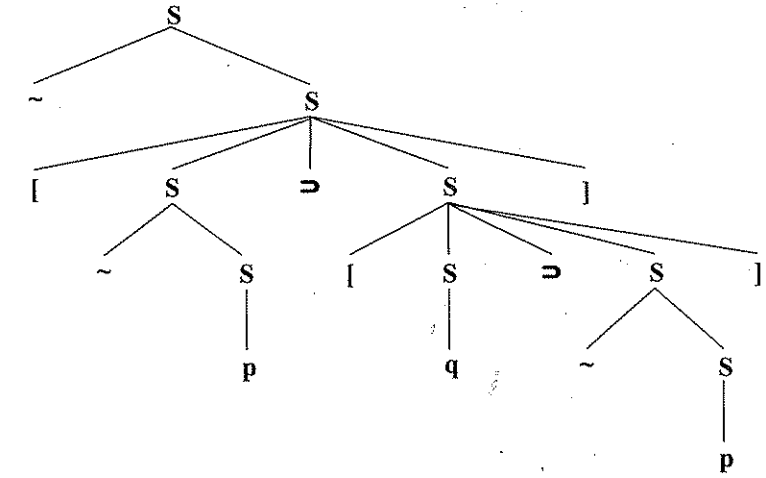
$$B_2 \Rightarrow B_3A_3$$

$$A_3 \Rightarrow \supset$$

$$B_3 \Rightarrow A_4S$$

$$A_4 \Rightarrow [$$

$$w = \sim[\sim p \supset [q \supset \sim p]]$$

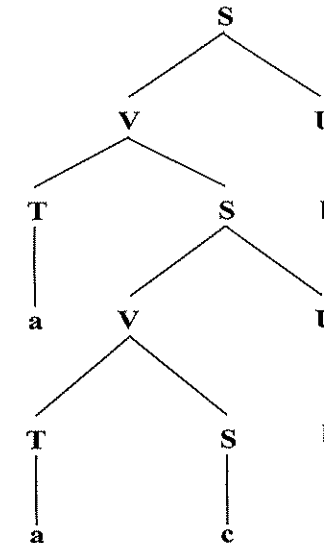


Y.4.12.

a) $G_{S.4.12}$ bağlamdan-bağımsız, Chomsky normal biçiminde (CNF) bir dilbilgisidir.

b) $L(G_{S.4.12}) = \{a^n c b^n \mid n \geq 0\}$

c) $w = aacbb$



Y.4.13.

a) $L_{S.4.13.1}$ bağlamdan-bağımsız bir dildir.

$$G_{S.4.13.1} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow aSb$$

$$A \Rightarrow aAb \mid \lambda$$

b) $L_{S.4.13.2}$ düzgün bir dildir.

$$G_{S.4.13.2} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow aA$$

$$A \Rightarrow aA \mid bB \mid b$$

$$B \Rightarrow bB \mid b$$

c) $L_{S.4.13.3}$ bağlamdan-bağımsız bir dildir.

$$G_{S.4.13.3} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow AB$$

$$A \Rightarrow 0A0 \mid 1A1 \mid \lambda$$

$$B \Rightarrow 0B0 \mid 1B1 \mid \lambda$$

d) $L_{S.4.13.4}$ bağlamdan-bağımsız bir dildir.

$$G_{S.4.13.4} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0S0 \mid 1S1 \mid \lambda$$

Y.4.14. $G_{S.4.14} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0SA \mid 1A \mid 0SBA \mid 1BA \mid 0$$

$$A \Rightarrow 0S \mid 1 \mid 0SB \mid 1B$$

$$B \Rightarrow 0SS \mid 1S \mid 0SBS \mid 1BS \mid 0SSB \mid 1SB \mid$$

$$0SBSB \mid 1BSB$$

Y.4.15. $G_{S.4.15} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, V, T\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow VT$$

$$V \Rightarrow aVb \mid ab$$

$$T \Rightarrow cT \mid c$$

Y.4.16.

a) $G_{S.4.16}$ bağlamdan-bağımsız bir dilbilgisidir ve Chomsky normal biçimindedir.b) $L(G_{S.4.16}) = \{a^n c b^n b^m c a^m \mid n, m \geq 0\}$

c) $S \Rightarrow TU \Rightarrow VBU \Rightarrow ATBU \Rightarrow aTBU \Rightarrow aVBBU \Rightarrow aATBBU$
 $\Rightarrow aaTBBU \Rightarrow aacBBU \Rightarrow aacbBU \Rightarrow aacbbU \Rightarrow aacbbWA$
 $\Rightarrow aacbbBUA \Rightarrow aacbbhUA \Rightarrow aacbbbcA \Rightarrow aacbbbca$

Y.4.25.

$$a) G_{S.4.25} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A\}$$

$$V_T = \{0, 1, 2\}$$

$$P: S \Rightarrow SS \mid A$$

$$A \Rightarrow 0A0 \mid 1A1 \mid 2A2 \mid 00 \mid 11 \mid 22$$

$$b) S \Rightarrow SS$$

$$\Rightarrow AS$$

$$\Rightarrow 0A0S$$

$$\Rightarrow 01A10S$$

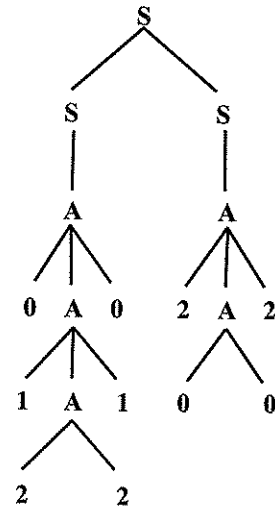
$$\Rightarrow 012210S$$

$$\Rightarrow 012210A$$

$$\Rightarrow 0122102A2$$

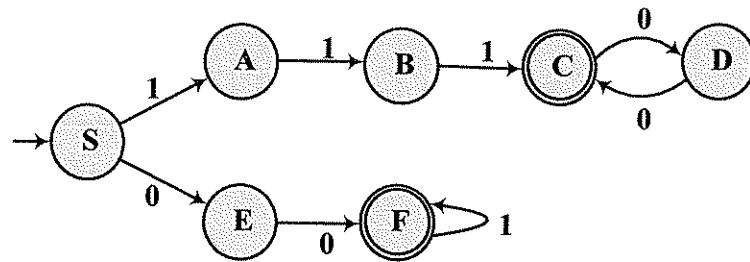
$$\Rightarrow 0122102002$$

Türetme Ağacı



Y.4.26.

a)

b) Geçiş çizeneği λ -geçiş içermemektedir.

c) Çizenekte yararsız durum yoktur.

d) Çizenekte tek başlangıç durumu vardır.

$$e) L_{S.4.26} = 111(00)^* + 001^*$$

Bölüm 5

Y.5.1.

$$a) G_{S.5.1} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow AB$$

$$A \Rightarrow aAb \mid ab$$

$$B \Rightarrow bBc \mid bc$$

b) PDA'nın çalışma ilkesi:

- a'lar yığta atılır.
- a'lar bitinceye kadar her b için yığtadan bir a silinir.
- a'lar bittikten sonra kalan b'ler yığta atılır.
- her c için yığtadan bir b silinir.

PDA'nın biçimsel tanımı:

$$M_{S.5.1} = \langle Q, \Sigma, \Gamma, q_0, \delta, Z_0, \Phi \rangle$$

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{a, b, c\}$$

$$\Gamma = \{Z_0, a, b\}$$

$$F = \Phi$$

$$\delta: \delta(q_0, a, Z_0) = (q_1, aZ_0)$$

$$\delta(q_1, a, a) = (q_1, aa)$$

$$\delta(q_1, b, a) = (q_2, \lambda)$$

$$\delta(q_2, b, a) = (q_2, \lambda)$$

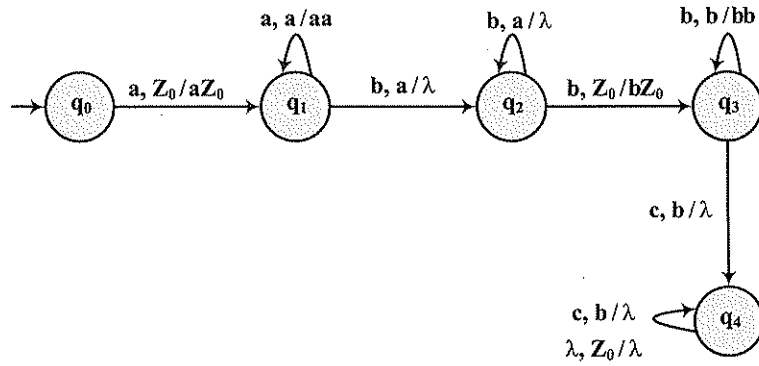
$$\delta(q_2, b, Z_0) = (q_3, bZ_0)$$

$$\delta(q_3, b, b) = (q_3, bb)$$

$$\delta(q_3, c, b) = (q_4, \lambda)$$

$$\delta(q_4, c, b) = (q_4, \lambda)$$

$$\delta(q_4, \lambda, Z_0) = (q_4, \lambda)$$



Y.5.2.

a) PDA'nın çalışma ilkesi:

- a'lar yığığa atılır.
- her b için yığıttan bir a silinir.
- her c için yığıttan bir a silinir.

b) $M_{S.5.2} = \langle Q, \Sigma, \Gamma, q_0, \delta, Z_0, \Phi \rangle$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b, c\}$$

$$\Gamma = \{Z_0, a, b, c\}$$

$$F = \Phi$$

$$\delta: \delta(q_0, a, Z_0) = (q_0, aZ_0)$$

$$\delta(q_0, a, a) = (q_0, aa)$$

$$\delta(q_0, b, a) = (q_1, \lambda)$$

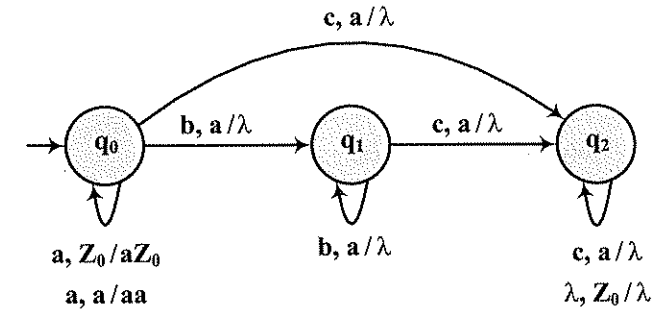
$$\delta(q_0, c, a) = (q_2, \lambda)$$

$$\delta(q_1, b, a) = (q_1, \lambda)$$

$$\delta(q_1, c, a) = (q_2, \lambda)$$

$$\delta(q_2, c, a) = (q_2, \lambda)$$

$$\delta(q_2, \lambda, Z_0) = (q_2, \lambda)$$



Y.5.3.

a) PDA'nın çalışma ilkesi:

- a'lar okunduğunda yığıtta hiçbir işlem yapılmaz.
- her b okunduğunda, yığıtta:
 - birden çok c varsa, bunlardan ikisi silinir,
 - tek bir c varsa, bu c silinir ve yığıta bir b eklenir,
 - hiç c yoksa, yığıta iki b eklenir.
- her c okunduğunda, yığıtta:
 - en az bir b varsa, bunlardan biri silinir,
 - hiç b yoksa, yığıta bir c eklenir.

PDA'nın biçimsel tanımı:

$$M_{S.5.3} = \langle Q, \Sigma, \Gamma, q_0, \delta, Z_0, \Phi \rangle$$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b, c\}$$

$$\Gamma = \{Z_0, b, c\}$$

$$F = \Phi$$

$$\delta: \delta(q_0, a, Z_0) = (q_1, Z_0)$$

$$\delta(q_0, b, Z_0) = (q_1, bbZ_0)$$

$$\delta(q_0, c, Z_0) = (q_1, cZ_0)$$

$$\delta(q_1, a, Z_0) = (q_1, Z_0)$$

$$\delta(q_1, a, b) = (q_1, b)$$

$$\delta(q_1, a, c) = (q_1, c)$$

$$\delta(q_1, b, Z_0) = (q_1, bbZ_0)$$

$$\delta(q_1, b, b) = (q_1, bbb)$$

$$\delta(q_1, b, c) = (q_2, \lambda)$$

$$\delta(q_1, c, Z_0) = (q_1, cZ_0)$$

$$\delta(q_1, c, b) = (q_1, \lambda)$$

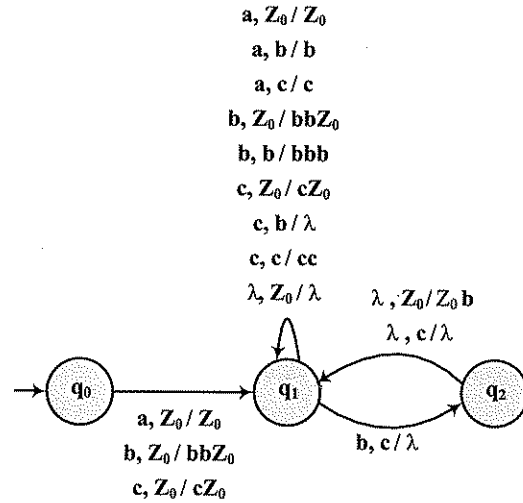
$$\delta(q_1, c, c) = (q_1, cc)$$

$$\delta(q_2, \lambda, Z_0) = (q_1, b)$$

$$\delta(q_2, \lambda, c) = (q_1, \lambda)$$

$$\delta(q_1, \lambda, Z_0) = (q_1, \lambda)$$

b)



$M_{S.5.3}$ deterministik değildir.

c) $G_{S.5.3} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B, C, D\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow aA \mid bCC \mid cD$$

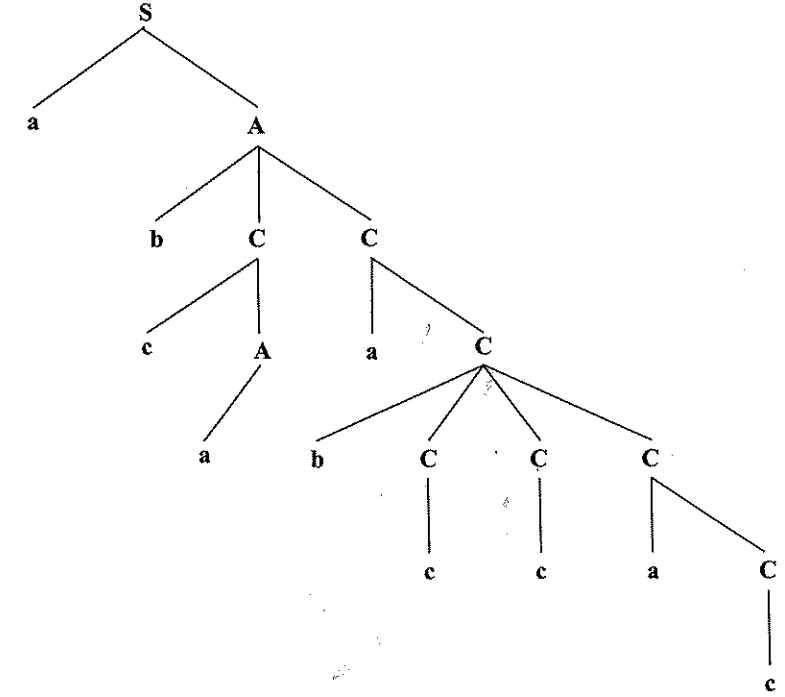
$$A \Rightarrow aA \mid bCC \mid cD \mid a \mid \lambda$$

$$B \Rightarrow aB \mid bA \mid cBD \mid b$$

$$C \Rightarrow aC \mid bCCC \mid cA \mid c$$

$$D \Rightarrow aD \mid bC \mid cB$$

$w = abcaabccac$ için türetme ağacı:



Y.5.4.

a) PDA'nın çalışma ilkesi:

- Yığıt boşsa:
 - a okunursa yığita A eklenir,
 - b ya da c okunursa yığita B eklenir.
- Yığıtın üstünde A varsa:
 - a okunursa yığita A eklenir,
 - b ya da c okunursa yığıttan A silinir.
- Yığıtın üstünde B varsa:
 - a okunursa yığıttan B silinir,
 - b ya da c okunursa yığita B eklenir.